# Mathematical methods for scientific computing
# Numerical discretisation

Stephen Millmore

Laboratory for Scientific Computing, University of Cambridge

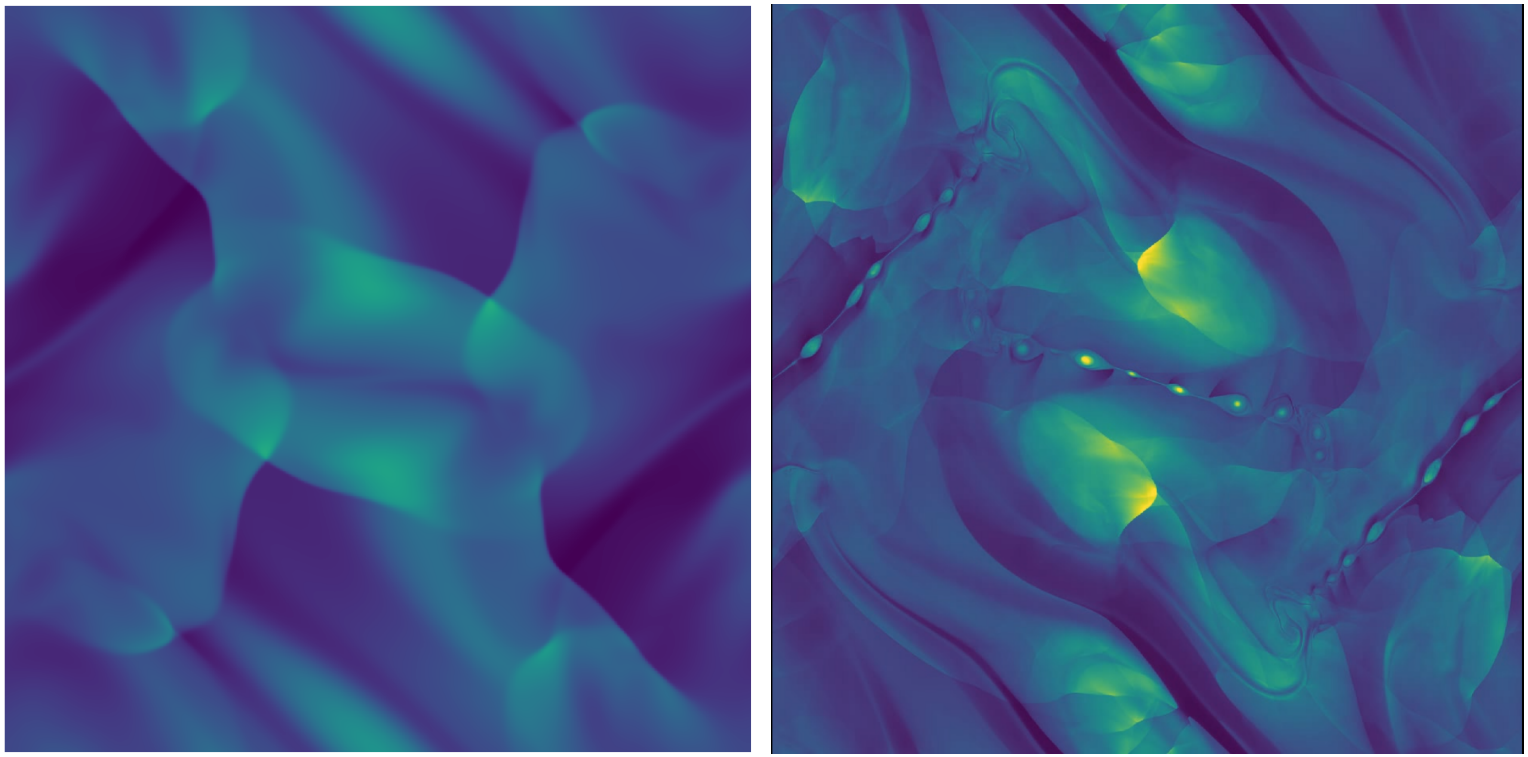
# Outline


1. Discretisation and error
2. Fourier Series (briefly)
3. Interpolation
4. Root finding
5. Runge-Kutta methods

# Discretisation

- All computational simulations require some means of discretisation of a continuum domain into a discrete one
- Any such discretisation will introduce an error
- Understanding this error, and how it will affect your simulations is key when employing a numerical method
- The various continuum courses will cover the error in specific methods in more detail, here we look at a few techniques for analysing error

# Effects of error



- Two different numerical methods attempting to simulate the same problem

# Order of accuracy

- All numerical methods have an **order of accuracy**; this is a measure of how quickly a numerical simulation approaches the true mathematical solution to the problem
- Taylor series are a good way to identify order of accuracy
- The Taylor series of a function $f(x)$ is computed about a point, $x = a$, as

$$f(x) = f(a) + \left.\frac{\partial f}{\partial x}\right|_{x=a} (x-a) + \left.\frac{1}{2}\frac{\partial^2 f}{\partial x^2}\right|_{x=a} (x-a)^2 + \left.\frac{1}{3!}\frac{\partial^3 f}{\partial x^3}\right|_{x=a} (x-a)^3 + \ldots$$

- In some cases, increasing the number of terms in the Taylor series gives a function that gets increasingly close to $f(x)$ (**it converges**)
- In many cases, this does not hold in general, but might be expected to give a reasonable approximation for $x$ close to $a$
- And we can use this to define approximations to derivatives

# Derivative approximations

- We can consider a Taylor expansion at a point $x + \Delta x$ about $x$,

$$f(x + \Delta x) = f(x) + \left.\frac{\partial f}{\partial x}\right|_x \Delta x + \left.\frac{\partial^2 f}{\partial x^2}\right|_x \frac{(\Delta x)^2}{2} + \left.\frac{\partial^3 f}{\partial x^3}\right|_x \frac{(\Delta x)^3}{3!} + \dots$$

- This approximation assumes that $\Delta x$ is sufficiently small to be convergent
- In terms of numerically solving a PDE, we may well know a value of $f$ at both $x$ and $x + \Delta x$, and it is the derivatives we need to calculate
- From this, we get

$$\left.\frac{\partial f}{\partial x}\right|_x = \frac{f(x + \Delta x) - f(x)}{\Delta x} + \left.\frac{\partial^2 f}{\partial x^2}\right|_x \frac{(\Delta x)}{2} + \left.\frac{\partial^3 f}{\partial x^3}\right|_x \frac{(\Delta x)^2}{3!} + \dots$$

# First-order derivative

- As $\Delta x$ is supposed to be small, it is reasonable to assume that any term multiplied by this is negligible
- For this reason, the derivative approximation is often written

$$\left.\frac{\partial f}{\partial x}\right|_x = \frac{f(x+\Delta x) - f(x)}{\Delta x} + \mathcal{O}(\Delta x)$$

- The $\mathcal{O}(\Delta x)$ term is the **error** within this approximation, and since $\Delta x > (\Delta x)^2 > (\Delta x)^3 > \ldots$ (again, for small $\Delta x$), this is the dominant term in the error
- Since $\Delta x = (\Delta x)^1$, this is referred to as a **first order accurate approximation**
- If we half $\Delta x$, we halve the error from that dominant term

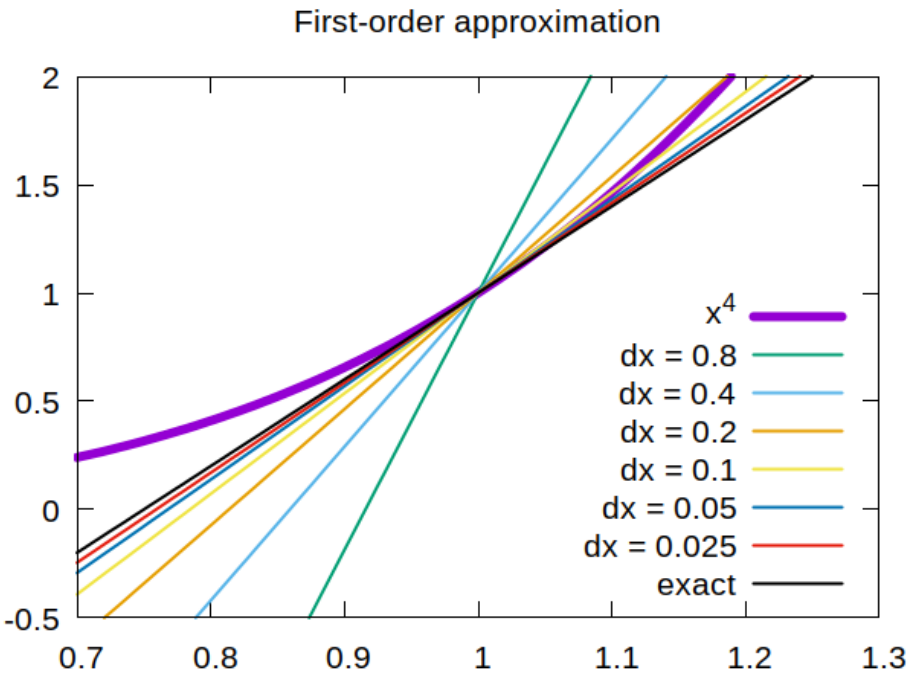
# Second-order derivative

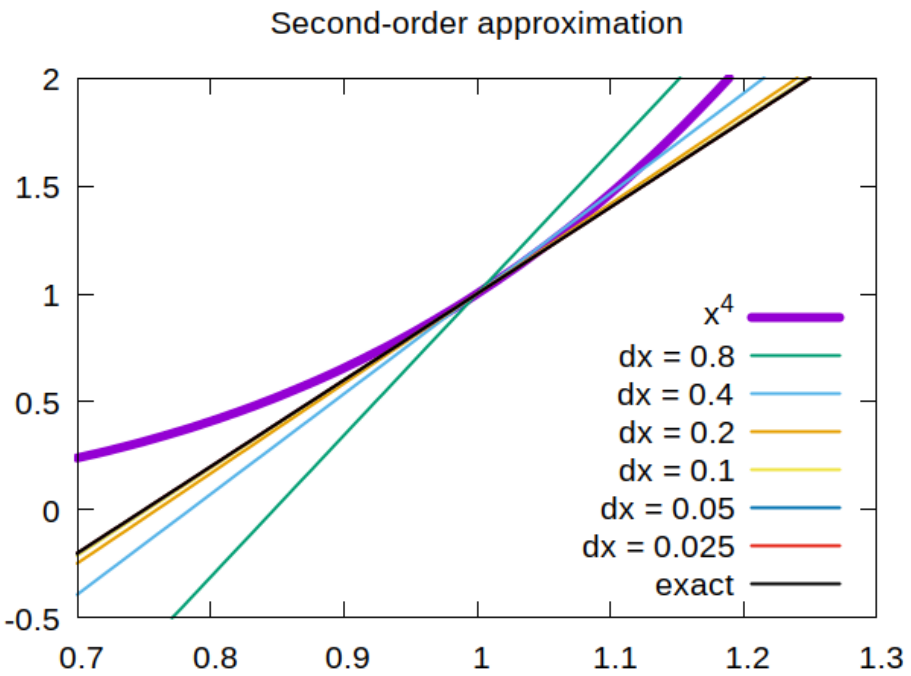
- The first order approximation used two values, $f(x)$ and $f(x + \Delta x)$, to compute the derivative
- It is, perhaps, unsurprising then that this is a relatively low-accuracy approximation
- We could do better by considering more points to compute the derivative
- Taylor expansions of $f(x + \Delta x)$ and $f(x - \Delta x)$ combine to give

$$\left.\frac{\partial f}{\partial x}\right|_x = \frac{f(x + \Delta x) - f(x - \Delta x)}{2\Delta x} + \mathcal{O}\left(\Delta x^2\right)$$

- This is a **second order accurate approximation**
- If we halve $\Delta x$, the error from the dominant term decreases by a factor 4

# Approximating derivatives



# Quantifying order of accuracy

- When approximating the derivative, we could consider that we know the true derivative, $g(x) = \frac{\partial f}{\partial x}$, and an approximate value, $\hat{g}(x, \Delta x)$
- If we choose the value of $x$, e.g. $x = 1$ in the previous slide, then we can compute the **error** in our approximation,

$$E(\Delta x) = |g - \hat{g}(\Delta x)|$$

- If our approximation, $\hat{g}$, has **order of accuracy** $n_{\mathcal{O}}$, then the error obeys

$$E(\Delta x) \leq C(\Delta x)^{n_{\mathcal{O}}}$$

- Or, if we don't want to try and find a constant, $C$,

$$\Delta E = \frac{E(\Delta x_1)}{E(\Delta x_2)} = \frac{C(\Delta x_1)^{n_{\mathcal{O}}}}{C(\Delta x_2)^{n_{\mathcal{O}}}} = \left(\frac{\Delta x_1}{\Delta x_2}\right)^{n_{\mathcal{O}}}$$

# Order of accuracy

$$\Delta E = \frac{E(\Delta x_1)}{E(\Delta x_2)} = \frac{C(\Delta x_1)^{n_\mathcal{O}}}{C(\Delta x_2)^{n_\mathcal{O}}} = \left(\frac{\Delta x_1}{\Delta x_2}\right)^{n_\mathcal{O}}$$

- From this, we have

$$n_\mathcal{O} = \frac{\ln(\Delta E)}{\ln\left(\frac{\Delta x_1}{\Delta x_2}\right)}$$

- Typically, we would choose $\Delta x_2 = 2\Delta x_1$, meaning

$$n_\mathcal{O} = -\frac{\ln(\Delta E)}{\ln 2}$$

- We can try this for our derivative example, where we have

$$\left.\frac{\partial f}{\partial x}\right|_{x=1} = 4x^3\big|_{x=1} = 4$$

$$\left.\frac{\partial \hat{f}_1}{\partial x}\right|_{x=1}(\Delta x) = \frac{(1+\Delta x)^4 - 1}{\Delta x}, \qquad \left.\frac{\partial \hat{f}_2}{\partial x}\right|_{x=1}(\Delta x) = \frac{(1+\Delta x)^4 - (1-\Delta x)^4}{2\Delta x}$$

# Order of accuracy - example

- For our first-order approximation we have

| $\Delta x$ | $\frac{\partial \hat{f}_1}{\partial x}\Big\vert_{x=1}(\Delta x)$ | $E_1(\Delta x)$ | $\Delta E_1(\Delta x)$ | $n_{\mathcal{O}}$ |
|---|---|---|---|---|
| 0.8 | 11.872 | 7.872 | - | |
| 0.4 | 7.104 | 3.104 | 0.3943 | 1.34 |
| 0.2 | 5.368 | 1.368 | 0.4407 | 1.18 |
| 0.1 | 4.641 | 0.641 | 0.4684 | 1.094 |
| 0.05 | 4.310125 | 0.310125 | 0.4838 | 1.047 |
| 0.025 | 4.1525 | 0.1525 | 0.4918 | 1.024 |

- Note that we may see regions where $n_{\mathcal{O}} > 1$, but overall, it is bounded by $n_{\mathcal{O}} = 1$

# Order of accuracy - example

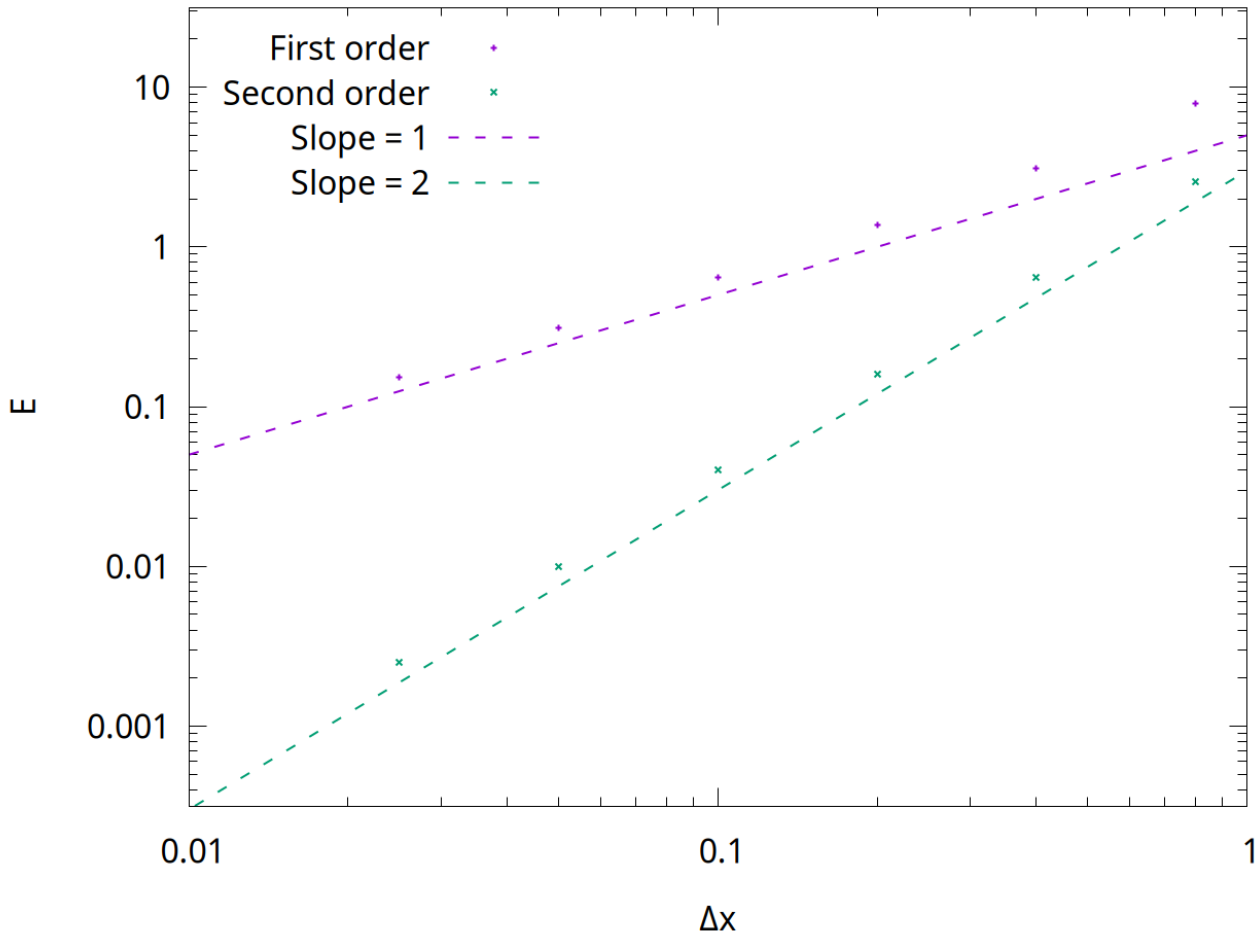
- For our second-order approximation we have

| $\Delta x$ | $\frac{\partial \hat{f}_2}{\partial x}\Big\|_{x=1}(\Delta x)$ | $E_2(\Delta x)$ | $\Delta E_2(\Delta x)$ | $n_{\mathcal{O}}$ |
|---|---|---|---|---|
| 0.8 | 6.56 | 2.56 | - | - |
| 0.4 | 4.64 | 0.64 | 0.25 | 2 |
| 0.2 | 4.16 | 0.16 | 0.25 | 2 |
| 0.1 | 4.04 | 0.04 | 0.25 | 2 |
| 0.05 | 4.01 | 0.01 | 0.25 | 2 |
| 0.025 | 4.0025 | 0.0025 | 0.25 | 2 |

- In this case, effectively perfect convergence is seen

# Order of accuracy - example

- Or we can plot this graphically



# Quantifying order of accuracy

- When solving partial differential equations, you do not have a single derivative to consider, but multiple terms in a coupled system
- And numerical methods for solving these equations do not always give you the answer you expect, this might be because:
  1. The errors from one derivative approximation dominate over others
  2. The method does not **converge** to a solution - it is unstable
  3. The tests you are using may not be as well behaved as the theory behind the method's analysis
  4. There is a bug in the code you are running
- In these cases, you want to use your numerical solution to obtain an order of accuracy of more than a single value, and still check that you are seeing the expected behaviour
- This is commonly done using $\mathbf{p}$-**norms** (or $\mathbf{l_p}$-**norms**)

# $p$-norms

- Norms are a way of measuring the size of a vector, and are a generalisation of absolute value
- One type of vector norm is very familiar (the Euclidean norm):

$$|\mathbf{a}| = (\mathbf{a} \cdot \mathbf{a})^{1/2} = \left(a_1^2 + a_2^2 + a_3^2\right)^{1/2}$$

- However, similar expressions exist for any length of vector, and the squaring and then taking the root can also be generalised
- If $\mathbf{b}$ is a vector with $N$ components, its $p$-norms are

$$||\mathbf{b}||_p = (|b_1|^p + |b_2|^p + |b_3|^p + \ldots + |b_N|^p)^{1/p}$$

- The idea for assessing order of accuracy of a numerical method is for $\mathbf{b}$ to be some measure of the error in a numerical solution compared to an exact solution

# Error norms

- Suppose that the $N$ components of $\mathbf{b}$ are the approximate solution to the function $B(x)$ computed at $N$ positions, $x = (x_1, x_2, \ldots, x_N)$
- For a given position, $x_i$, the error is given by

$$E_i = b_i - B(x_i)$$

- Effectively we want to sum up the error, but it needs to be appropriately weighted
- If each of the positions is equally distributed in space, the **error norms** are defined by

$$||\mathbf{E}||_p = \left( \frac{x_N - x_1}{N} \sum_{i=1}^{N} |E_i|^p \right)^{1/p}$$

# Common error norms

- In theory, all error norms provide a useful measure or error, but in practice, it is rare to find anyone using a 4-norm, or a 23-norm
- It is most common to see the 1-norm (particularly for nonlinear problems), 2-norm (particularly for linear problems) and $\infty$-norm (particularly for smooth problems)
- The $\infty$-norm is the limit of the error norms as $p \to \infty$
- In this limit:

$$\lim_{p\to\infty} \sum_{i=1}^{N} |E_i|^p = \max_i |E_i|^p \quad \text{and} \quad \lim_{p\to\infty} \left(\frac{x_N - x_1}{N}\right)^{1/p} = 1$$

- As a result,

$$||\mathbf{E}||_\infty = \max_i |E_i|$$

# Computing order of accuracy from error norms

- If error norms are computed for two different approximations, the ratio of these errors can be used to compute the order of accuracy
- In this case, the now have

$$||\Delta E||_p = \frac{||E(\Delta x_1)||_p}{||E(\Delta x_2)||_p} = \frac{C(\Delta x_1)^{n\mathcal{O}}}{C(\Delta x_2)^{n\mathcal{O}}} = \left(\frac{\Delta x_1}{\Delta x_2}\right)^{n\mathcal{O}}$$

- This is still easiest when we have $\Delta x_2 = 2\Delta x_1$
- In practice, multiple error norms are computed for a given method (e.g. repeatedly doubling the number of points) for confidence in the value - as was needed for our first-order derivative example
- So far, we have assumed that an exact solution exists too - if it doesn't, comparisons of e.g. $\mathbf{b}_1$ and $\mathbf{b}_2$ can be made with a third solution, $\mathbf{b}_3$, however, additional work may be required to show that there is **convergence to the correct solution**

# Outline


1. Discretisation and error
2. Fourier Series (briefly)
3. Interpolation
4. Root finding
5. Runge-Kutta methods

# Fourier series

- Error norms are a practical technique to show that a numerical solution is converging
- Fourier series can be used to provide a theoretical technique for finding when a numerical solution will converge
- Methods to actually do this will be found in other courses, we provide a brief overview of some important properties of Fourier series here
- Recall, Fourier series represent a **periodic function** as a sum of trigonometric functions

$$g(x) = A_0 + \sum_{n=1}^{\infty} \left( A_n \cos\left(\frac{2\pi n x}{P}\right) + B_n \sin\left(\frac{2\pi n x}{P}\right) \right)$$

# Computing Fourier series

- For a given function, $g(x)$, with periodicity, $P$, the constants of the Fourier series are given by

$$A_0 = \frac{1}{P}\int_{-P/2}^{P/2} g(x)\mathrm{d}x$$

$$A_n = \frac{2}{P}\int_{-P/2}^{P/2} g(x)\cos\left(\frac{2\pi n x}{P}\right)\mathrm{d}x$$

$$B_n = \frac{2}{P}\int_{-P/2}^{P/2} g(x)\sin\left(\frac{2\pi n x}{P}\right)\mathrm{d}x$$

- The idea behind using Fourier series in computing convergence, however, is not to compute these quantities, but to find when they are bounded
- If a solution is found on a finite domain, then by replicating this domain, it is straightforward to make it periodic
- We can express the solution, **or its error**, as a Fourier series

# Converging solutions

- Assume we have a Fourier series expansion for the error of a solution
- If we can show, for every $n$, that this error decreases as the simulation progresses, we know we will **converge to a solution**
- This is often possible to do simply, and analytically
- This requires an equivalent form of the Fourier series,

$$g(x) \approx \sum_{n=-\infty}^{\infty} \left( c_n \exp\left( \frac{2\pi i n x}{P} \right) \right)$$

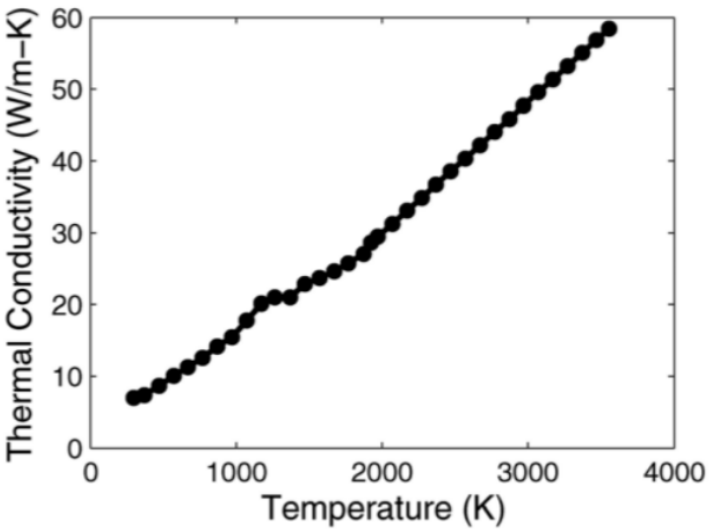
- The advantage of this form is that we only need to worry about a single $c_n$, which makes simplification easier

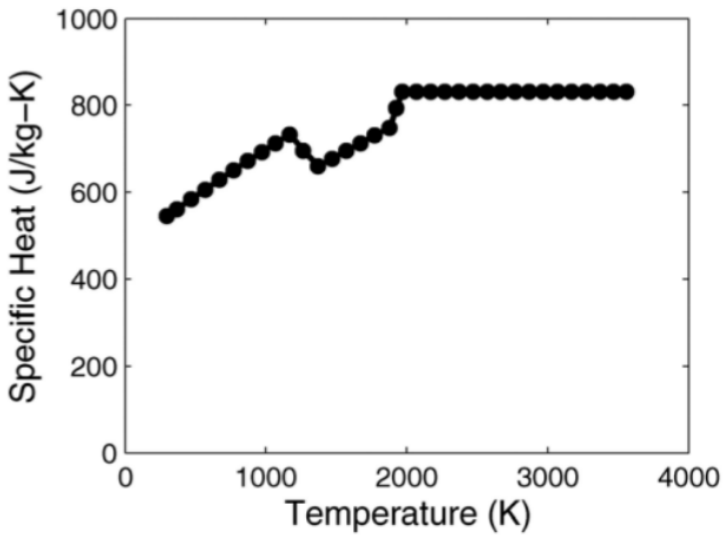
# Outline


1. Discretisation and error
2. Fourier Series (briefly)
3. Interpolation
4. Root finding
5. Runge-Kutta methods

# Discretisation and interpolation

- When we have a solution on discrete space, we may need information about points **in between** these known values
- This could be for identifying material interfaces, identifying contours or determining material properties




- To achieve this, we may find we need to interpolate data

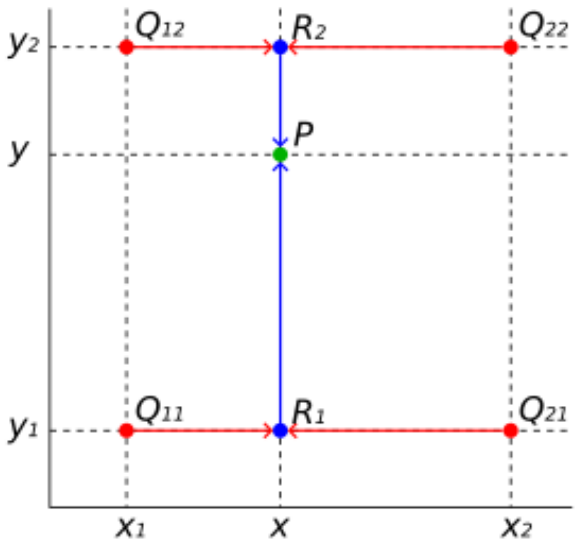
# Discretisation and interpolation

- Like with other numerical methods, different types of discretisation have different orders of accuracy (and different computational cost)
- In many cases, a simple linear interpolation is sufficient because:
  - There is a low computational cost for such interpolation
  - There is no issue with overshooting near the edges of the data range
  - Often, e.g. at material interfaces, accuracy is already low (covered in multiphysics lectures) - better interpolation gets us nothing
  - The tabulated data is sufficiently accurate that even first-order accurate interpolation has a low error
- For a function $y = y(x)$, if we know $(x_1, y_1)$ and $(x_2, y_2)$, and that $x_1 \leq x \leq x_2$, then linear interpolation gives

$$y(x) \approx y_1 + (y_2 - y_1) \frac{x - x_1}{x_2 - x_1}$$

# Bilinear and trilinear interpolation

- The use of simple interpolation functionality is more important if you are in multiple dimensions
- Bilinear and trilinear interpolation is the 2D and 3D equivalent to linear interpolation
- And they also work by considering one-dimension at a time
- In 2D, if we have $z = z(x, y)$




$$z(x, y_1) \approx \frac{x_2 - x}{x_2 - x_1} z(x_1, y_1) + \frac{x - x_1}{x_2 - x_1} z(x_2, y_1)$$

$$z(x, y_2) \approx \frac{x_2 - x}{x_2 - x_1} z(x_1, y_2) + \frac{x - x_1}{x_2 - x_1} z(x_2, y_2)$$

$$z(x, y) \approx \frac{y_2 - y}{y_2 - y_1} z(x, y_1) + \frac{y - y_1}{y_2 - y_1} z(x, y_2)$$

# Other interpolation

- Bilinear and trilinear interpolation work in very similar ways, taking one direction at a time
- The order of the direction is also unimportant
- If more accurate interpolation is needed, spline interpolation exists in 1D, bicubic in 2D and tricubic in 3D
- In order to get greater accuracy, however, more points are needed, 16 in the case of bicubic interpolation, and 64 in the case of tricubic

# Outline


1. Discretisation and error
2. Fourier Series (briefly)
3. Interpolation
4. Root finding
5. Runge-Kutta methods

# Iterative solutions in this course

- The other lecture courses within the MPhil will cover numerical techniques for complex systems of ODEs and PDEs
- In this course, we consider the techniques which could be classed as a numerical method, but are general, and found across a range of applications
- The first one we shall consider is root finding
- Many materials are governed by complex relationships between physical quantities such as internal energy ($\varepsilon$), density (related to $I_1$, $I_2$ and $I_3$) and entropy ($s$)

$$\varepsilon = \frac{K_0}{2\alpha_R^2}\left(I_3^{\alpha_R/2} - 1\right)^2 + c_V T_0 I_3^{\gamma_R/2}\left(e^{s/c_v} - 1\right) + \frac{b_0^2}{2} I_3^{\beta_R/2}\left(\frac{I_1^2}{3} - I_2\right)$$

- If we need to invert this equation for any reason (e.g. to obtain $I_3$, which is raised to two different powers), we may struggle to do it analytically
- Iterative techniques can help us

# Analytic, exact and approximate solutions

- Numerical methods that will be studied in the continuum courses are approximate solutions - these have a finite error and order of accuracy
- Analytic solutions are those that can be written down as an equation on paper - there is no inaccuracy in calculating a solution (in theory)

$$p = (\gamma - 1)\,\rho\varepsilon$$

- Exact solutions are those to which an equation exists defining the solution, but this could be implicit, or non-invertible (analytic solutions are a subset of this case)

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{t^2/2} \mathrm{d}t$$

- Iterative solutions (or root-finding techniques) are computational (or algorithmic) methods which can compute exact solutions to an arbitrary degree of accuracy

# Bisection method

- The bisection method is a reliable, but relatively inefficient, algorithm for finding the roots to an equation
- Whatever we are trying to solve needs to be written as

$$f(x) = 0$$

- For example,

$$f(I_3) = \varepsilon - \frac{K_0}{2\alpha_R^2}\left(I_3^{\alpha_R/2} - 1\right)^2 + c_V T_0 I_3^{\gamma_R/2}\left(e^{s/c_v} - 1\right) + \frac{b_0^2}{2} I_3^{\beta_R/2}\left(\frac{I_1^2}{3} - I_2\right) = 0$$

  where we may know $\varepsilon$, $s$, $I_1$ and $I_2$, and want to find the value of $I_3$
- To start a bisection root finding algorithm, we need two values; one where $f(I_3) > 0$, and one where $f(I_3) < 0$
- The bisection method is then guaranteed to find **one root** between these two values - this may not be the only root

# Bisection method

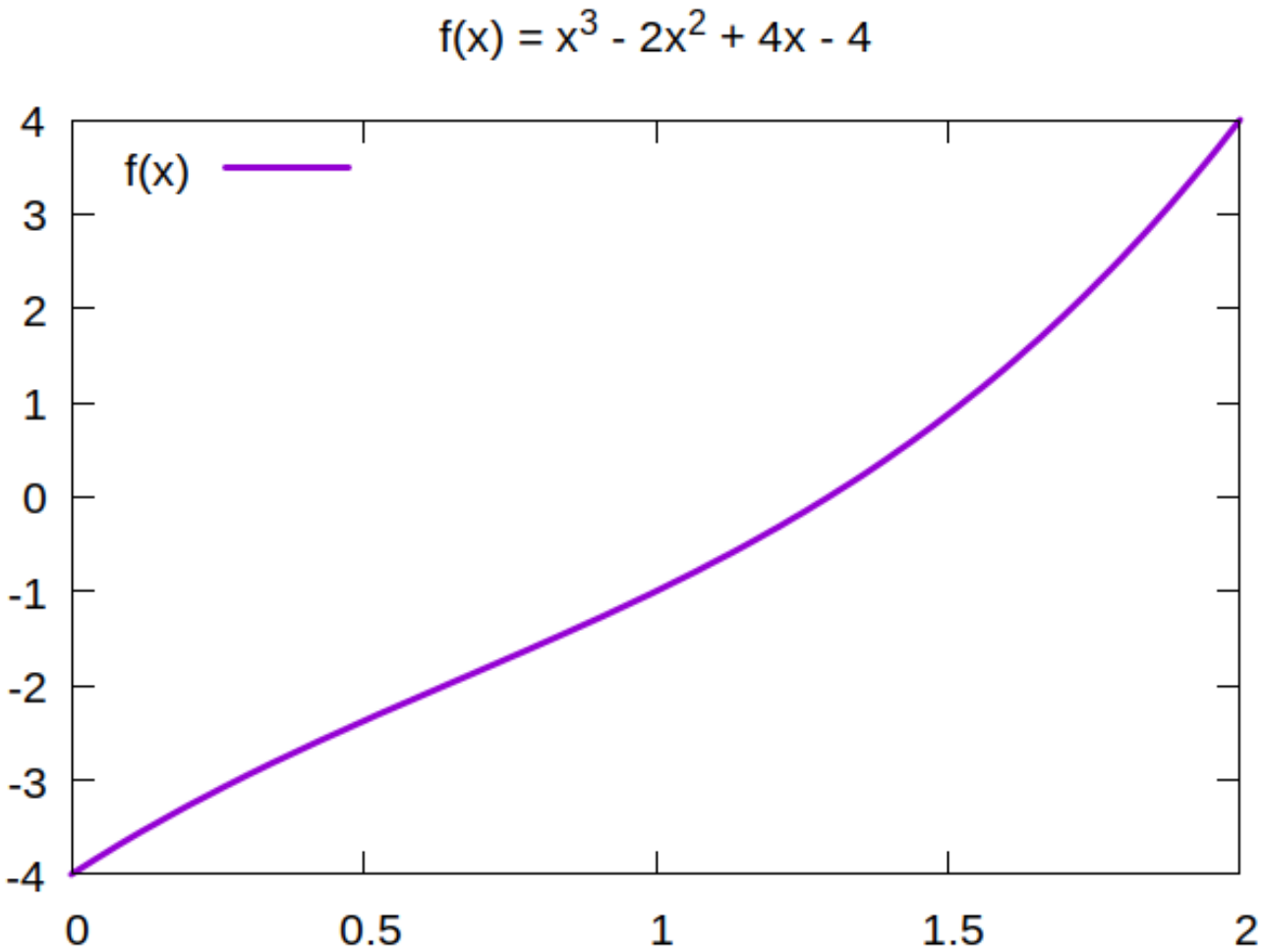
- Assume we have $a$ and $b$ such that $f(a) < 0$ and $f(b) > 0$

1. Set $c = \frac{1}{2}(a + b)$
2. Evaluate $f(c)$
3. If $|f(c)| < \epsilon$, or $\frac{|f(c)|}{c} < \epsilon$, where $\epsilon$ is some sufficiently small number, then $c$ is the root of the equation
4. Otherwise, $c$ will replace either $a$ or $b$

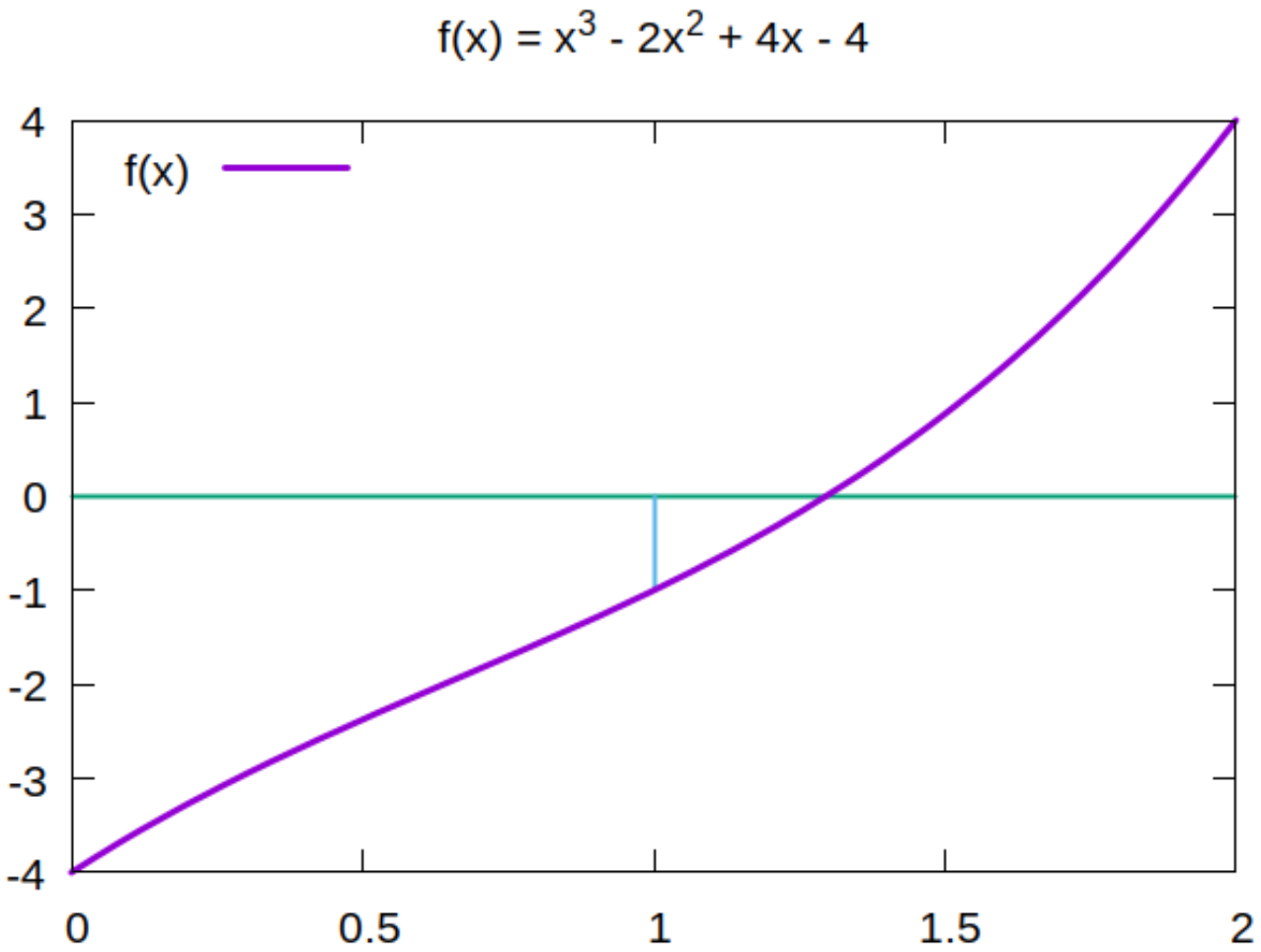
$$\begin{cases} f(c) < 0: & \text{set } a = c \\ f(c) > 0: & \text{set } b = c \end{cases}$$

5. Repeat until we have found the root of the equation

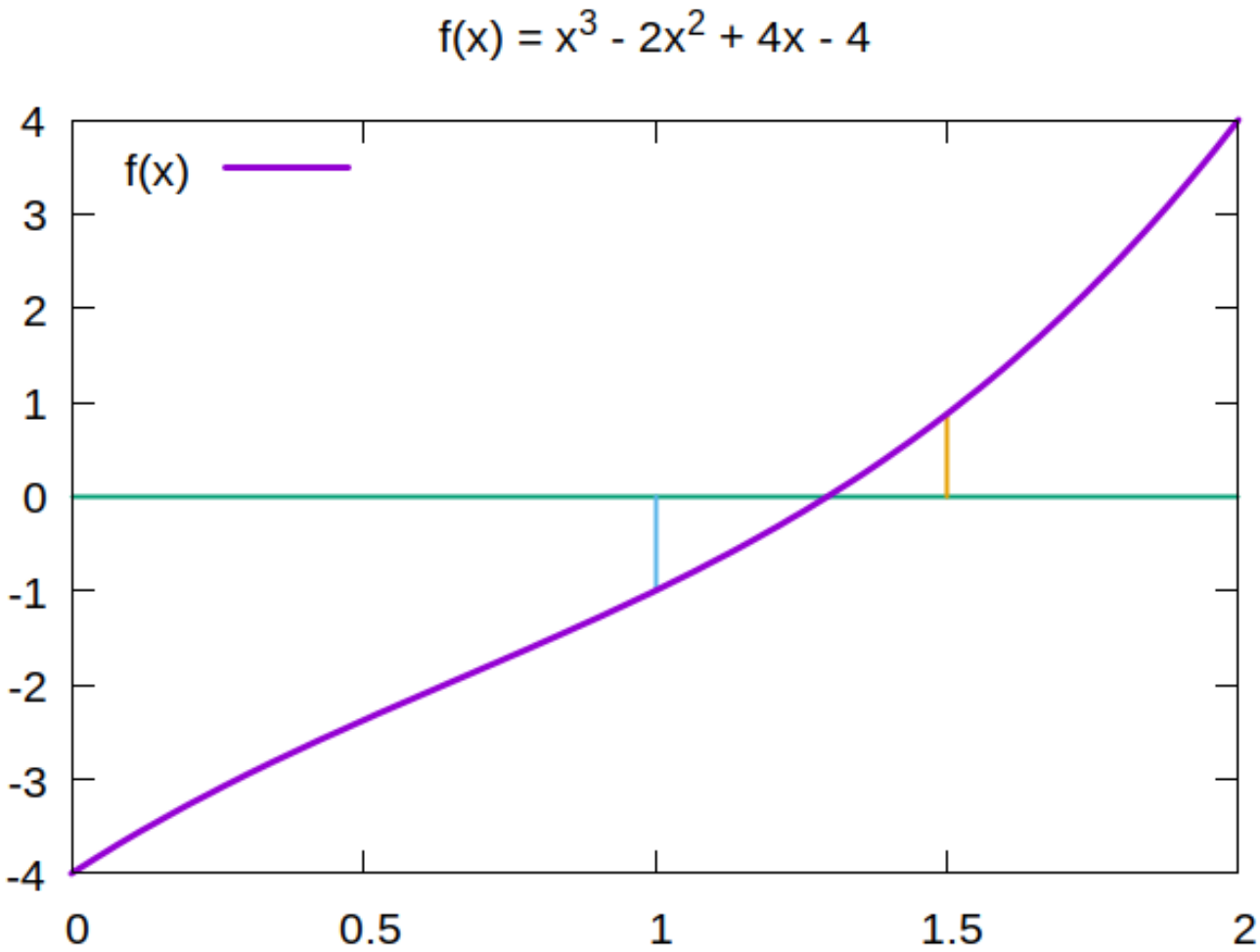
# Bisection method



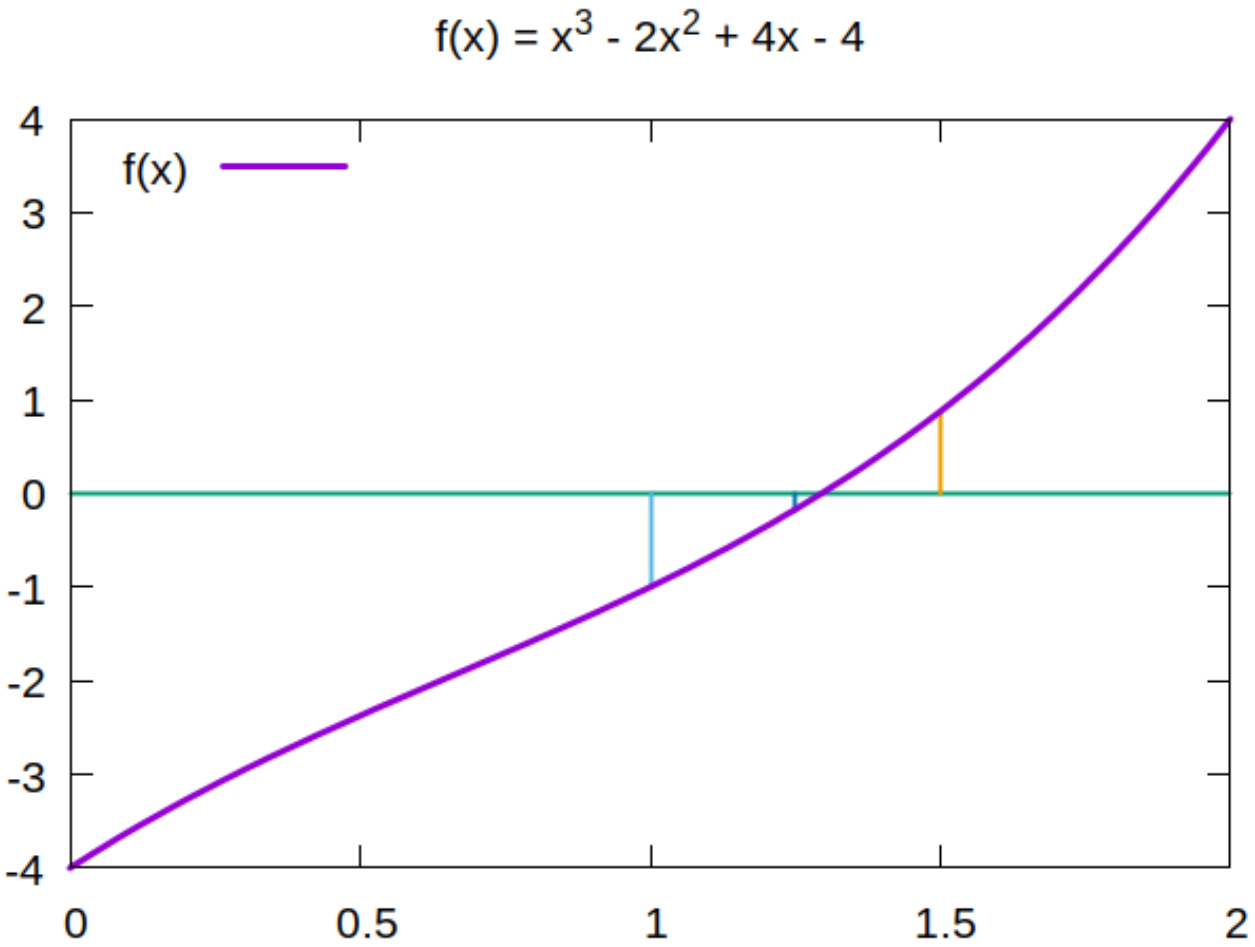
# Bisection method



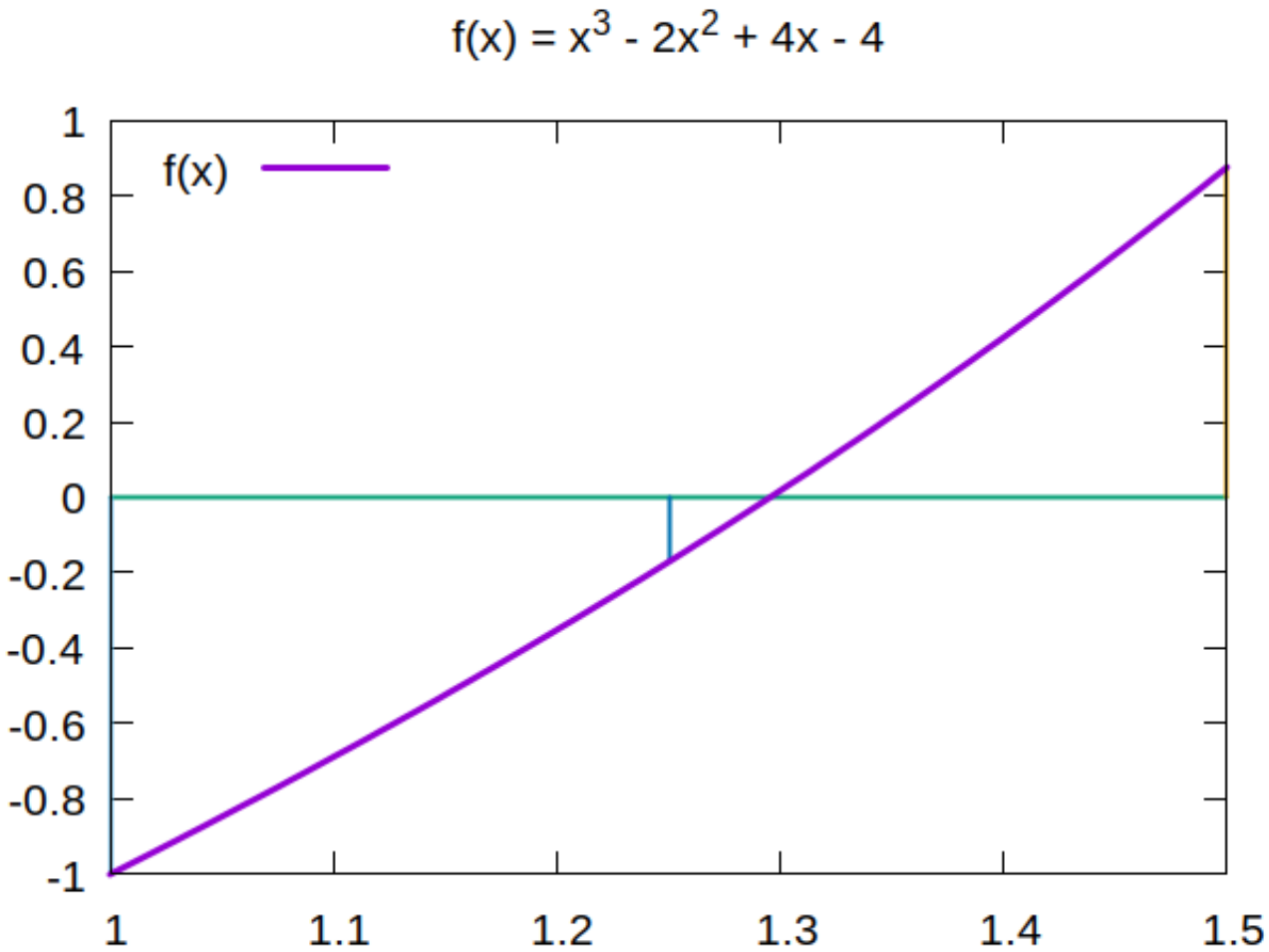
# Bisection method



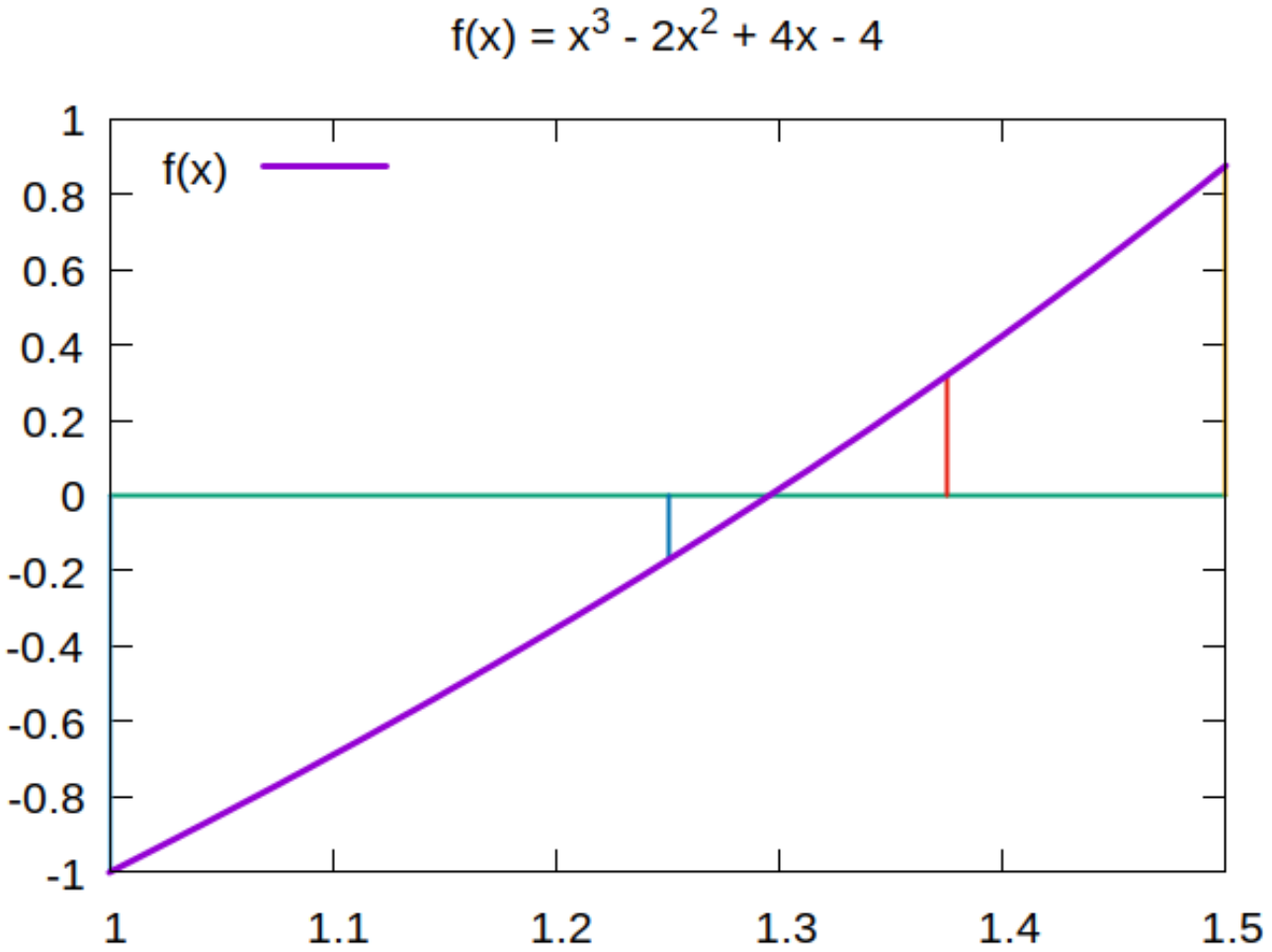
# Bisection method



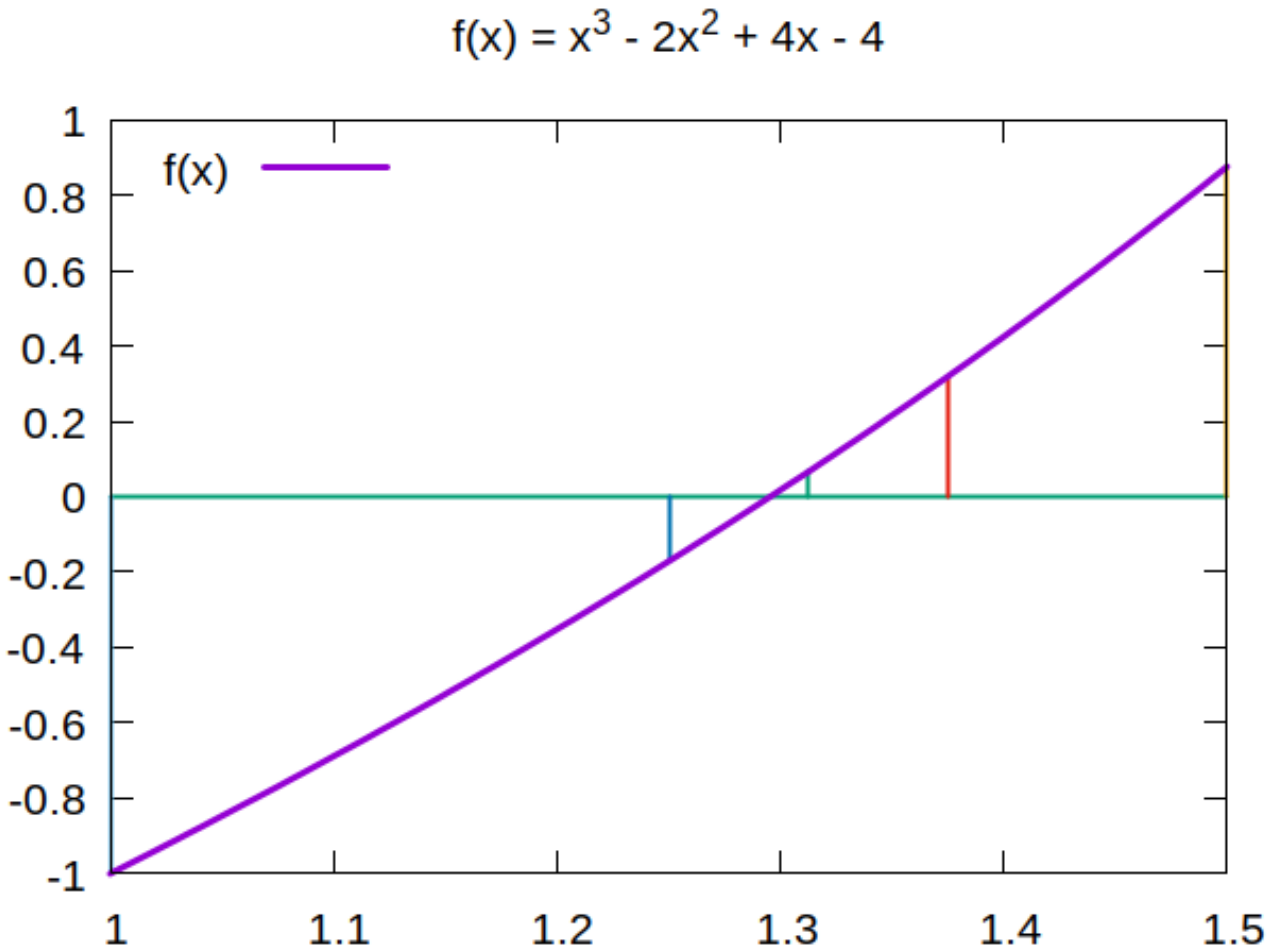
# Bisection method



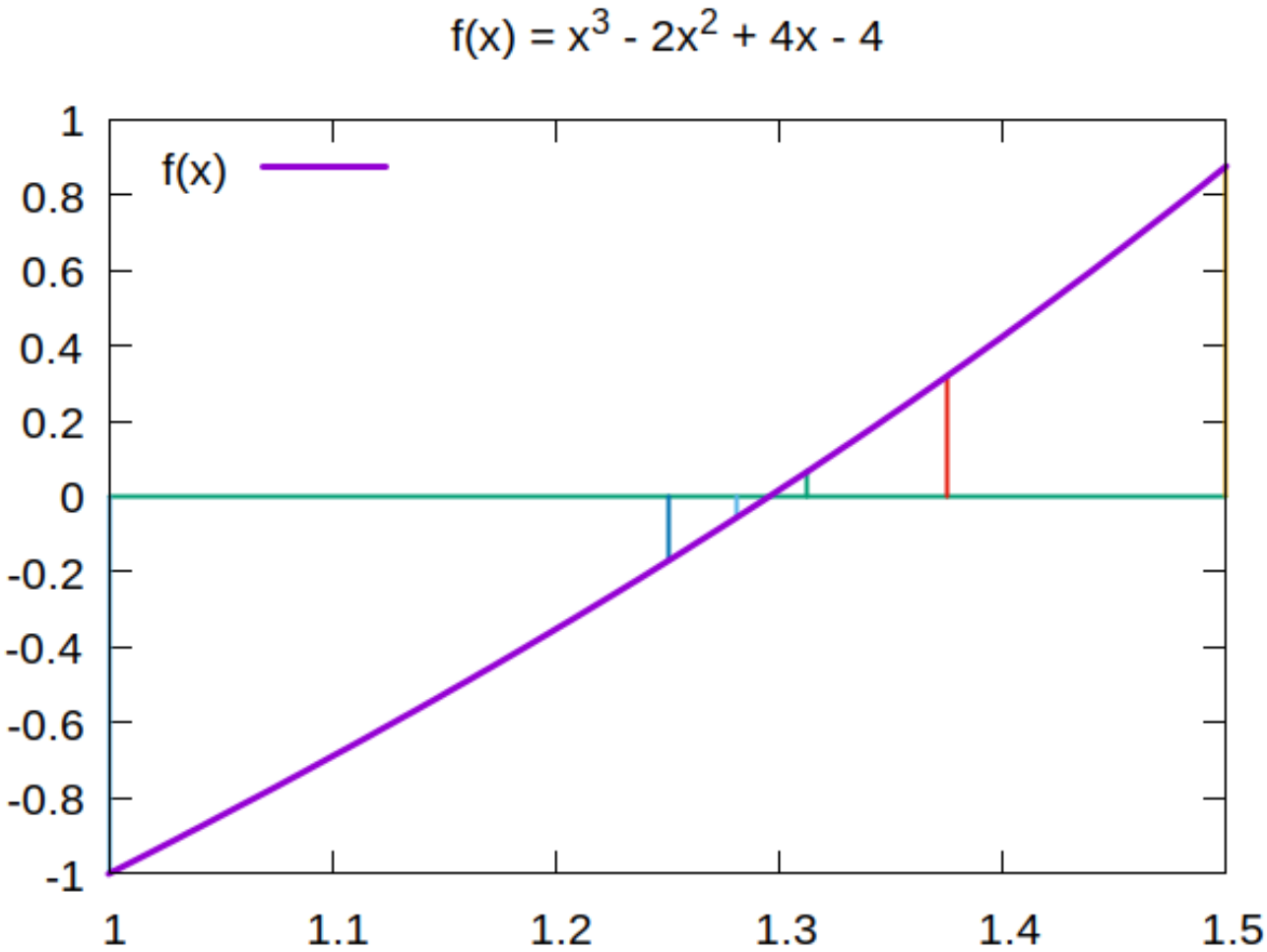
# Bisection method



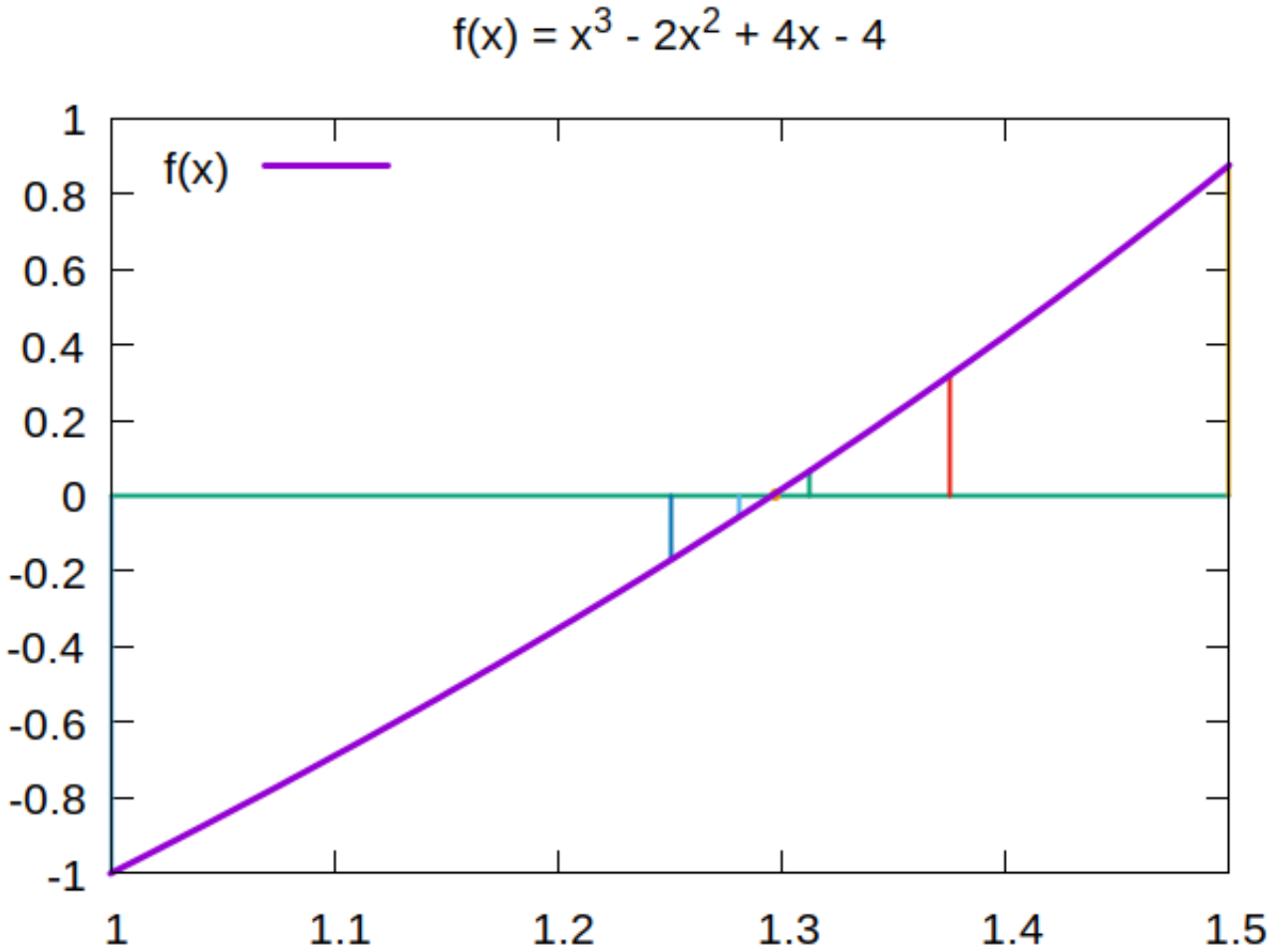
# Bisection method



# Bisection method

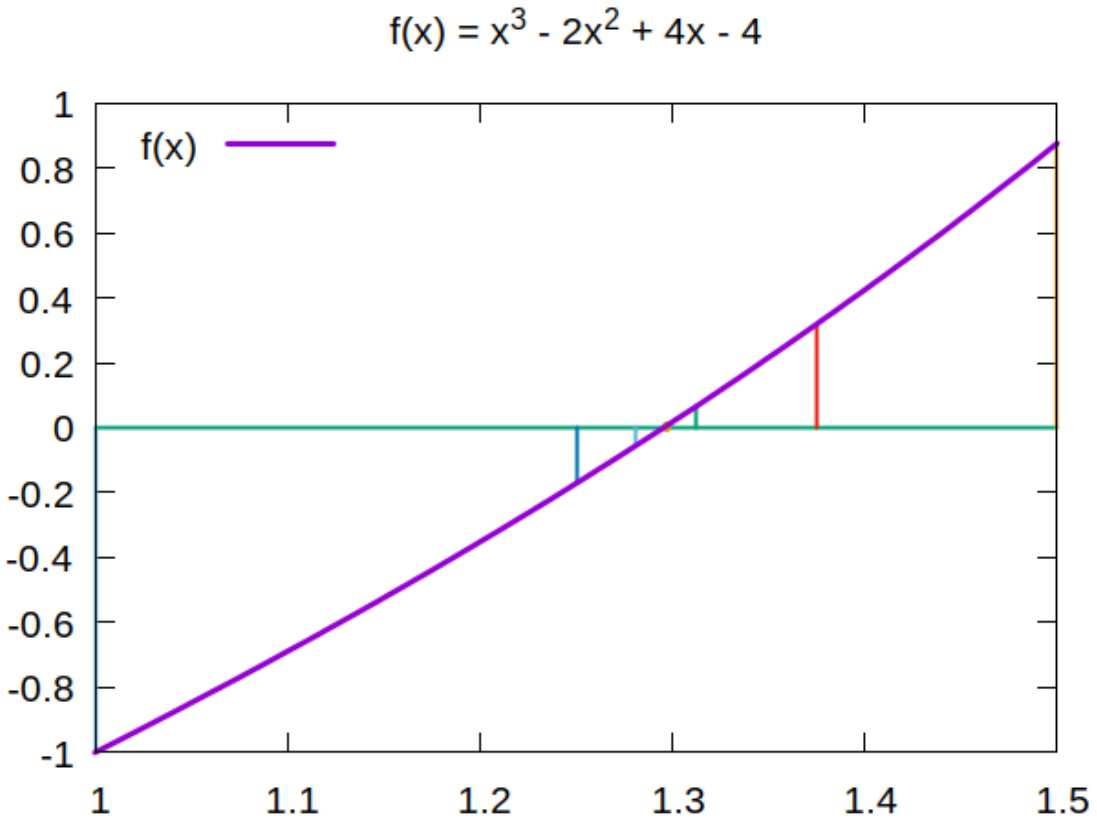


# Bisection method



# Bisection method

- After 6 iterations:
  $c = 1.296875$
  $f(c) = 0.004925$



# Notes on the bisection method

- The bisection method's guess does not always have to get closer to the root
- Generally, physical parameters are often 'well-behaved' - they increase or decrease **monotonically**
- But not always - in these cases, more physical information about the system may be required to ensure you have the correct root
- The method does require you to find two suitable guesses - these are often obvious, but problem dependent. You may have a starting guess from the previous time this method was called, for example

# Newton's method

- The bisection method is reliable, simple, but inefficient
- Under certain measures, it is the best method which is **guaranteed** to find a root
- In a numerical simulation, however, root finding may need to be done a lot - inefficient is not good
- Newton's method, which is efficient, simple, and usually reliable, is a popular choice to improve things
- The idea behind the method is that if, for any single guess, we follow the slope of the derivative down to the axis, we get closer to the solution

# Newton's method

- For Newton's method, we make an initial guess, $x_n$, for which we know $f(x_n)$
- For the method to work, we need to know the derivative of $f(x)$, and be able to calculate it at $x_n$
- We then find the point $x_{n+1}$ such that the for the tangent line, $t(x_{n+1}) = 0$
- Fortunately, this is straightforward, since, for a straight line

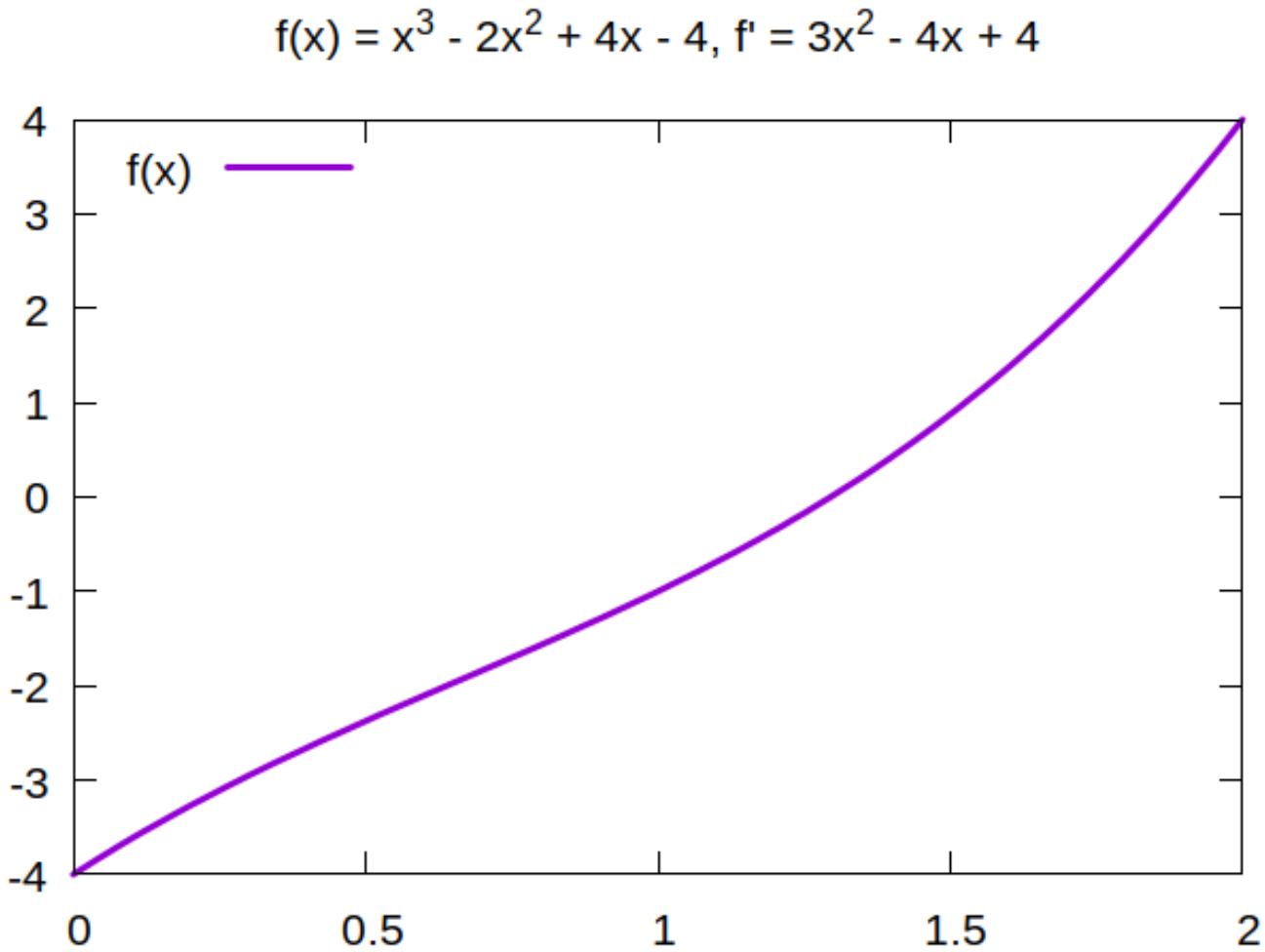
$$\left.\frac{\partial f}{\partial x}\right|_{x=x_n} = \frac{0 - f(x_n)}{x_{n+1} - x_n}$$

- Our new guess for $x$ is then

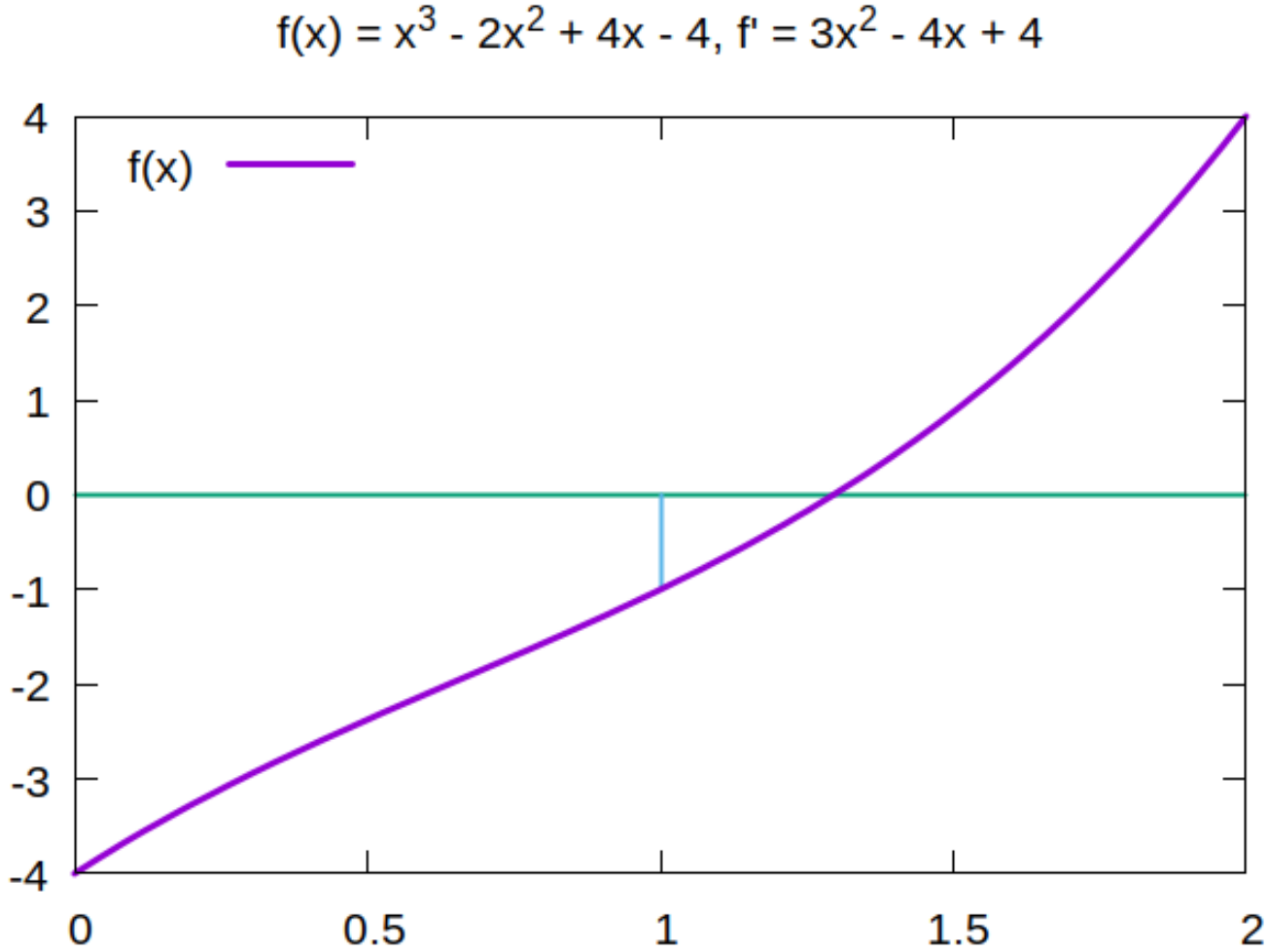
$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

- If $f(x_{n+1}) < \epsilon$ then we have converged
- Otherwise, repeat the above formula to get $x_{n+2}$ and continue until convergence

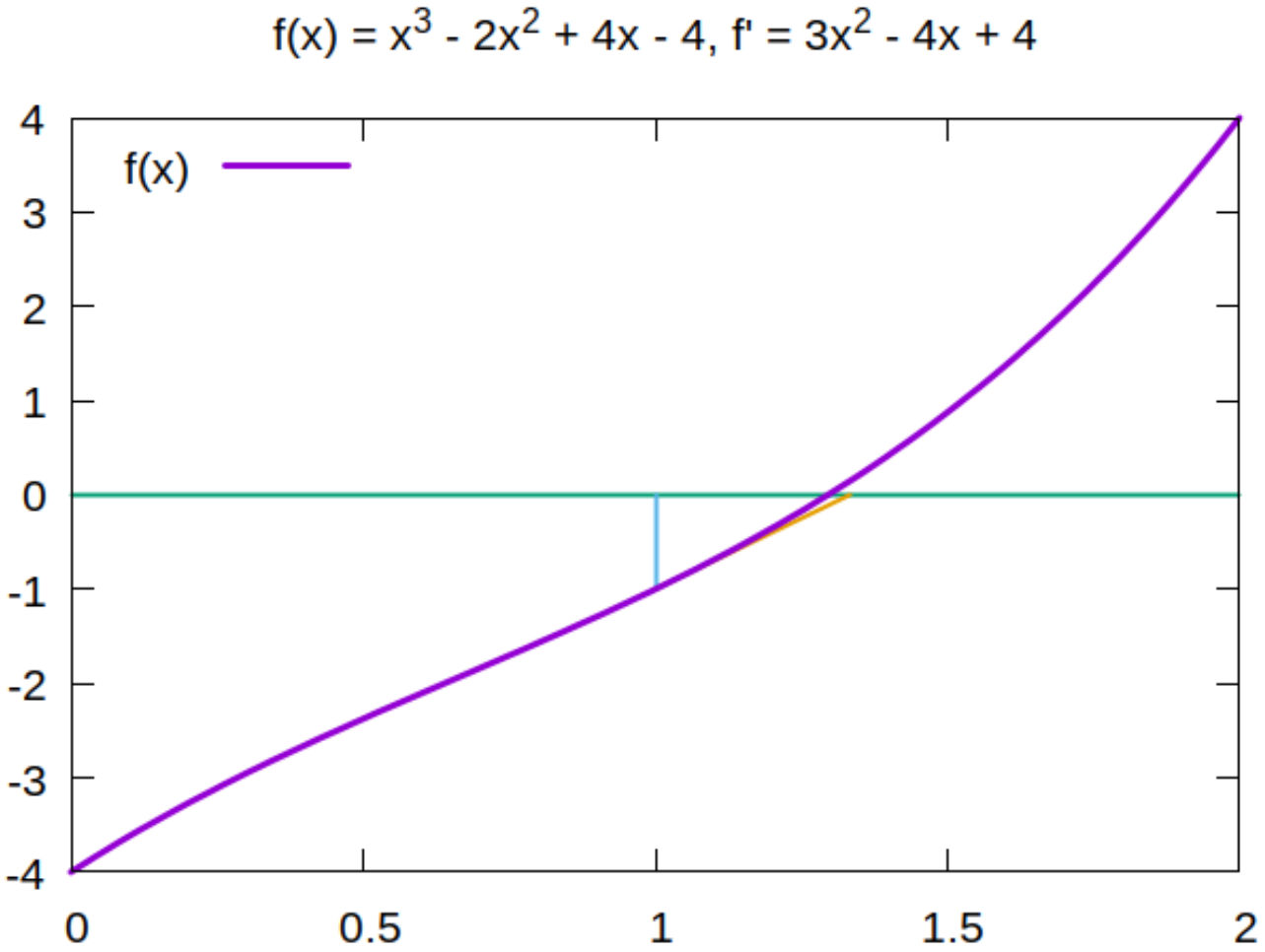
# Newton's method



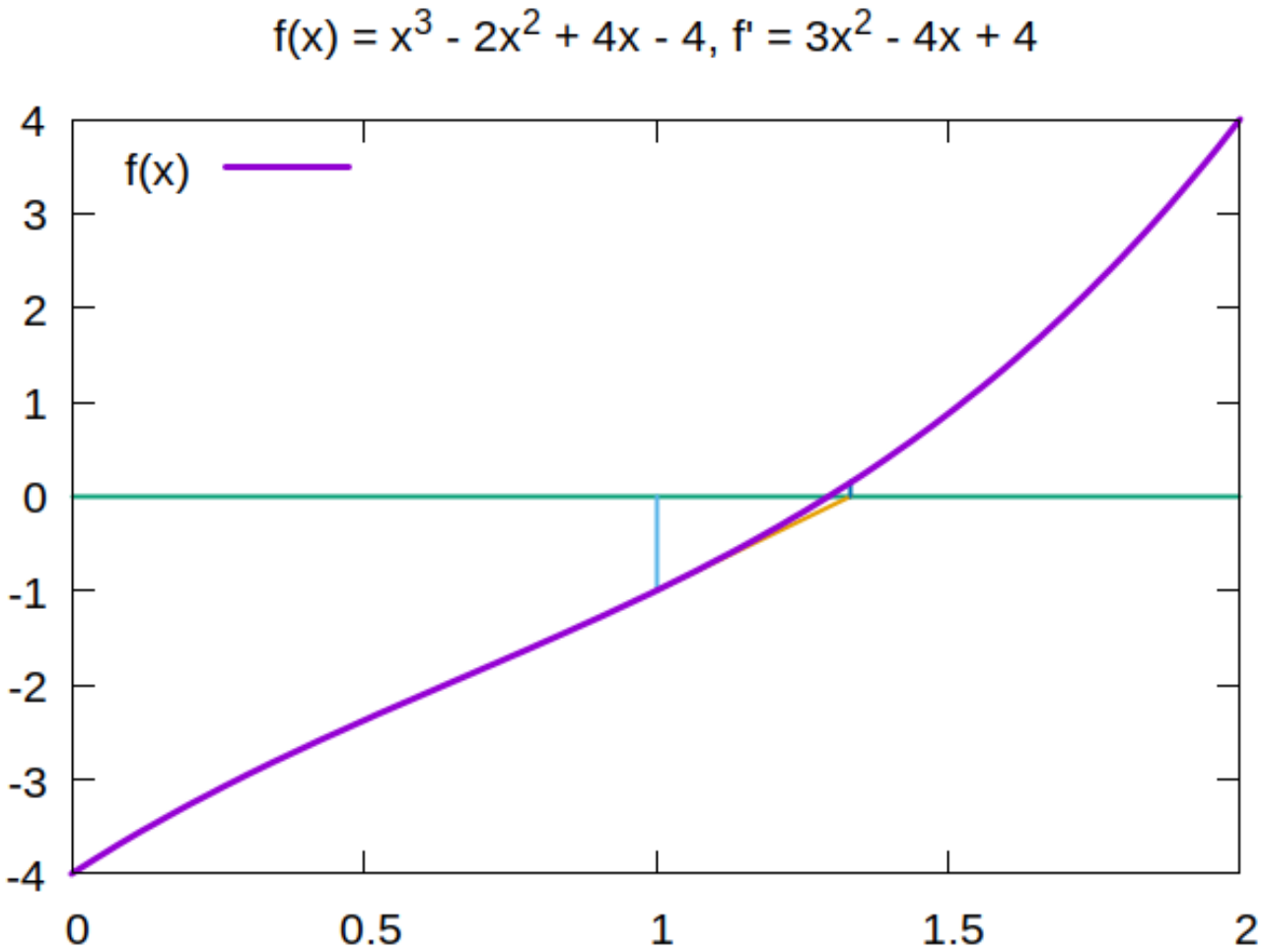
# Newton's method



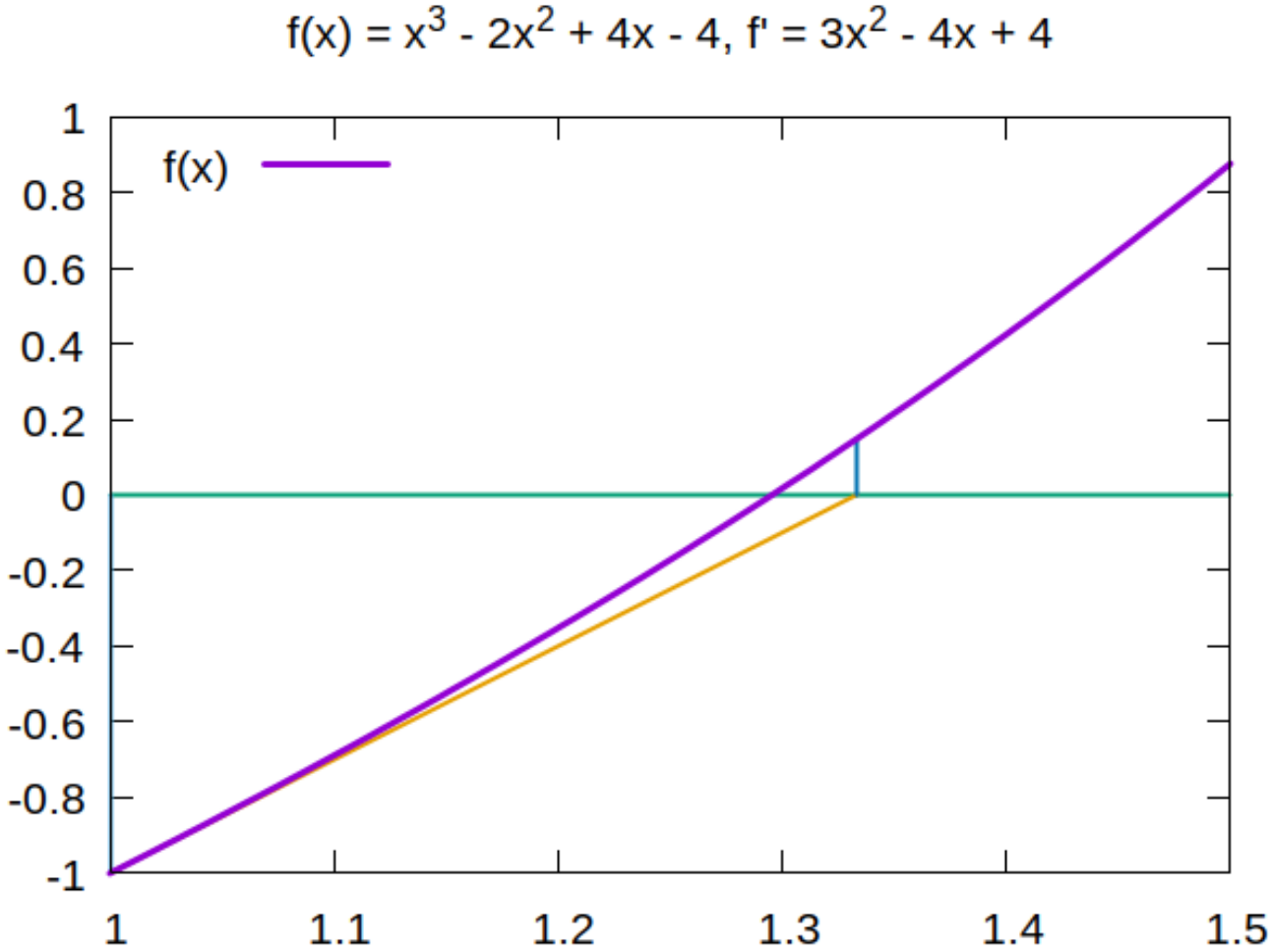
# Newton's method



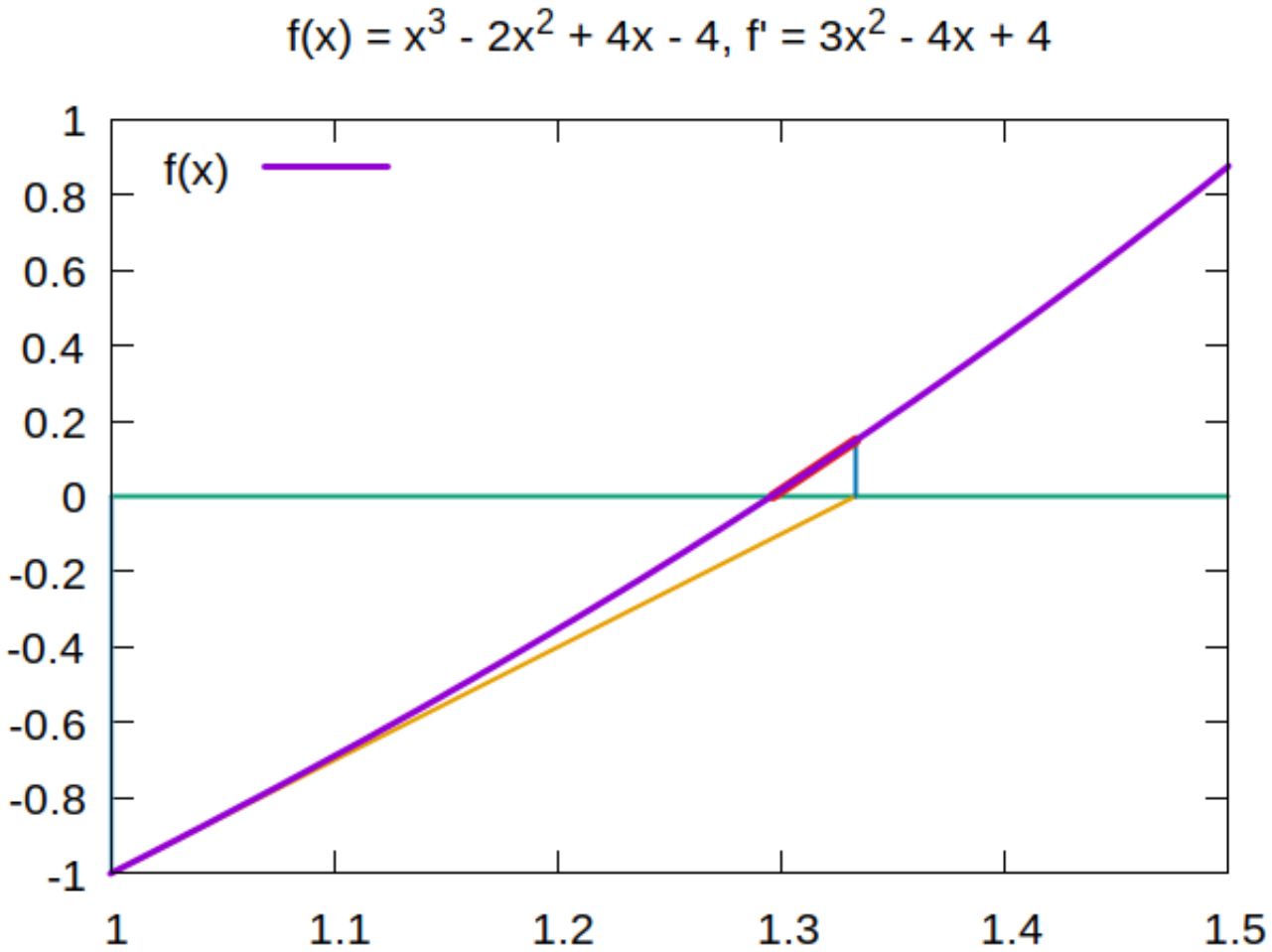
# Newton's method



# Newton's method

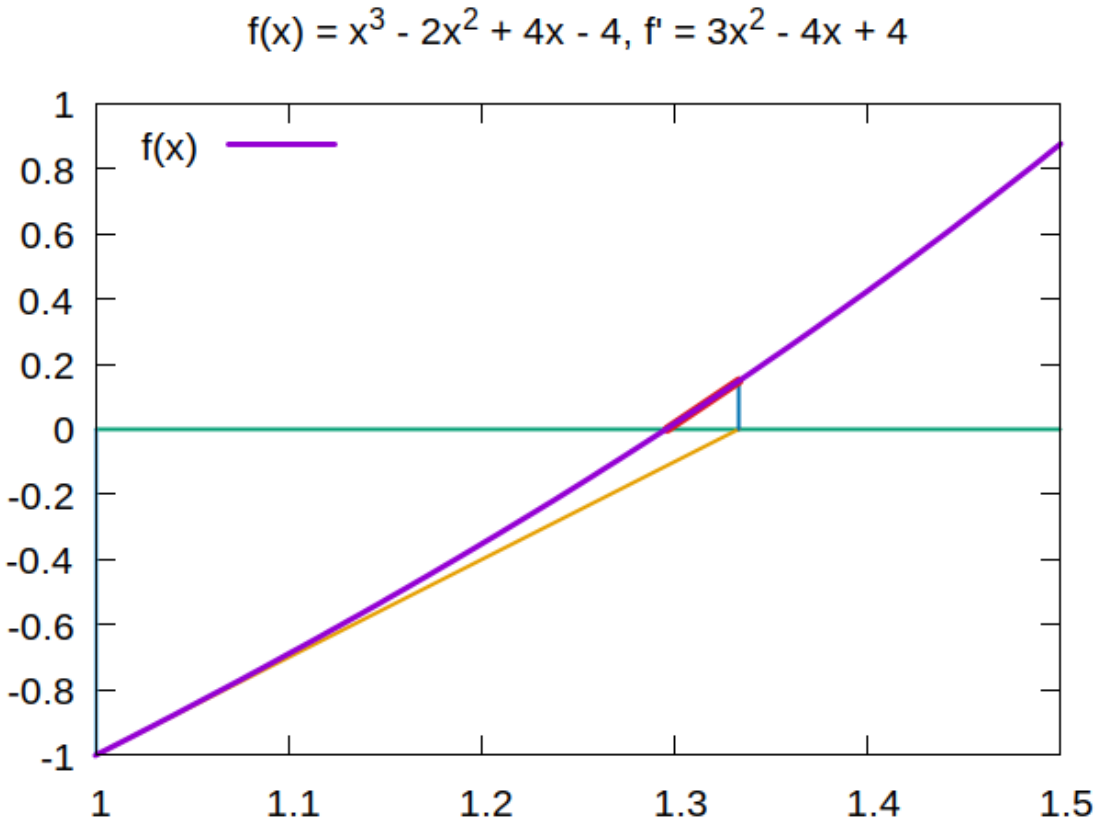


# Newton's method



# Newton's method

- After 2 iterations:
  $c = 1.2963$
  $f(c) = 0.002967$



# Newton's method

- When Newton's method works, it usually works quickly, but it can have issues
- Clearly, if the derivative of $f(x)$ doesn't exist, or if $f'(x_n) = 0$, the algorithm fails
- However, it is also possible for $x_{n+1}$ to end up further from the root than $x_n$ was, and for this cycle to continue
- This is most likely if the initial guess is too far from the true root
- Sometimes, convergence is just slow (e.g. if the root is a minimum or maximum of $f(x)$)
- Whilst there are sometimes techniques to deal with this behaviour, in practice, it is often easiest to try Newton's method, and if that fails, try something guaranteed to find the root, e.g. the bisection method
- Note that root finding methods do not know the physics of your system - the guesses may be outside a regime that is physically permissible, and equations may not hold

# Convergence of Newton's method

- When Newton's method does converge, it converges fast, and we can quantify this
- Consider a function $f(x)$ with a root at $\bar{x}$, i.e. $f(\bar{x}) = 0$
- The error of iteration $n+1$ is

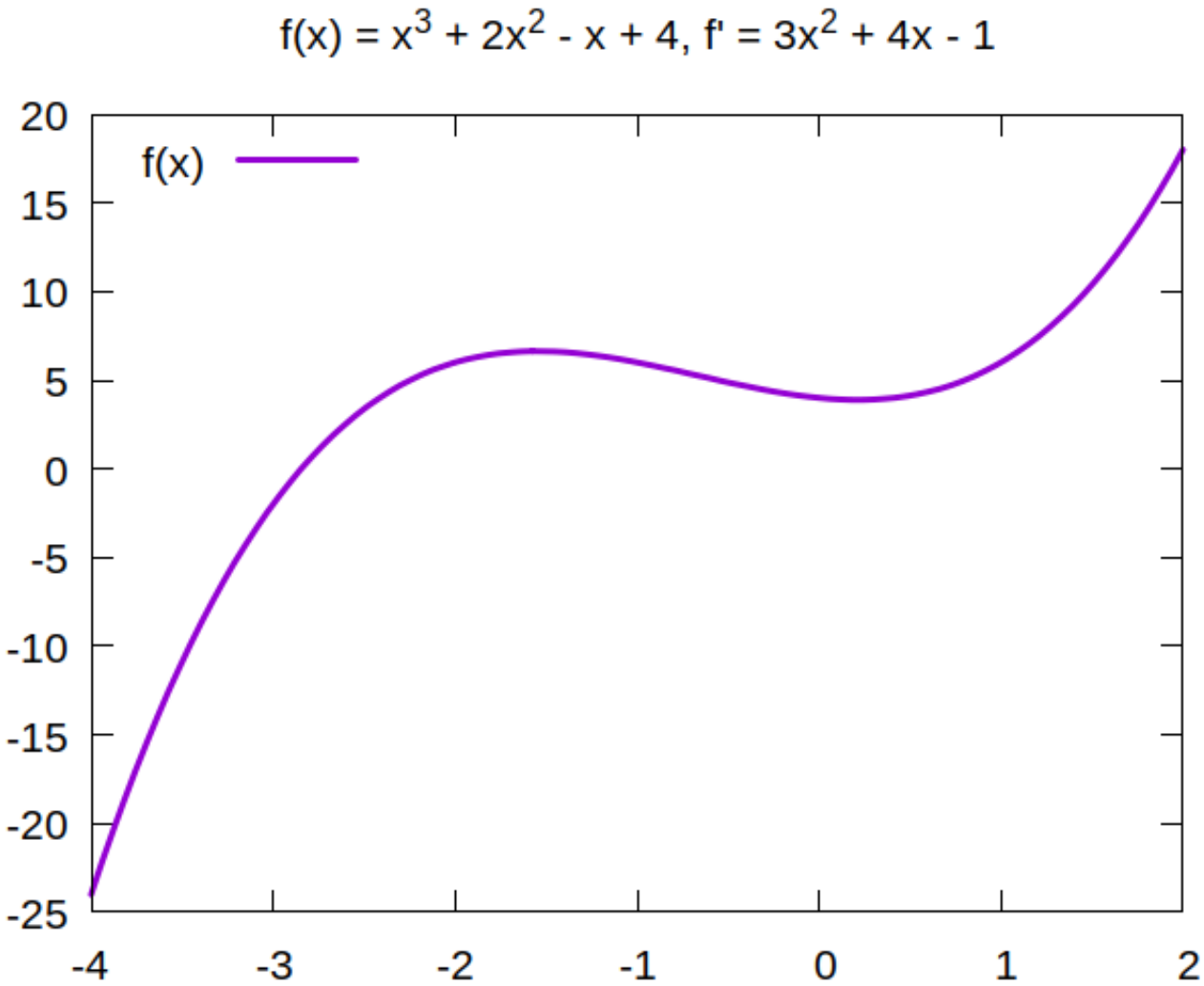
$$\epsilon_{n+1} = x_{n+1} - \bar{x}$$

- With a little manipulation of this expression, we can show that

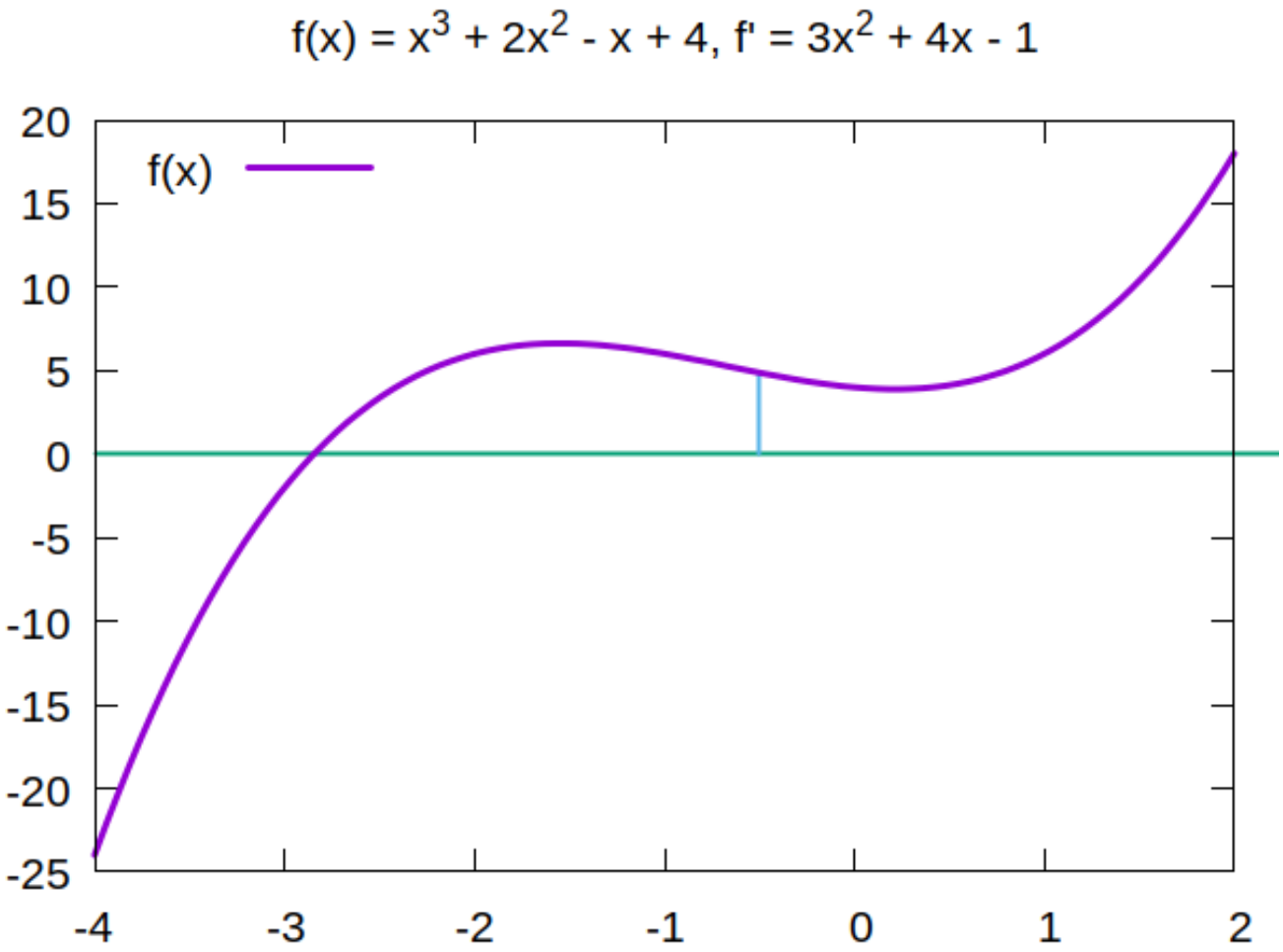
$$\epsilon_{n+1} = \frac{1}{2}\epsilon_n^2 \frac{f''(\bar{x})}{f'(\bar{x})} + \mathcal{O}\left(\epsilon_n^3\right)$$

- This is quadratic convergence; the error of the $(n+1)^{\text{th}}$ iteration is the square of the error of the $n^{\text{th}}$ iteration; provided $\epsilon_n$ is small, this error will decrease rapidly!
- But what if $\epsilon_n$ isn't small at some point?

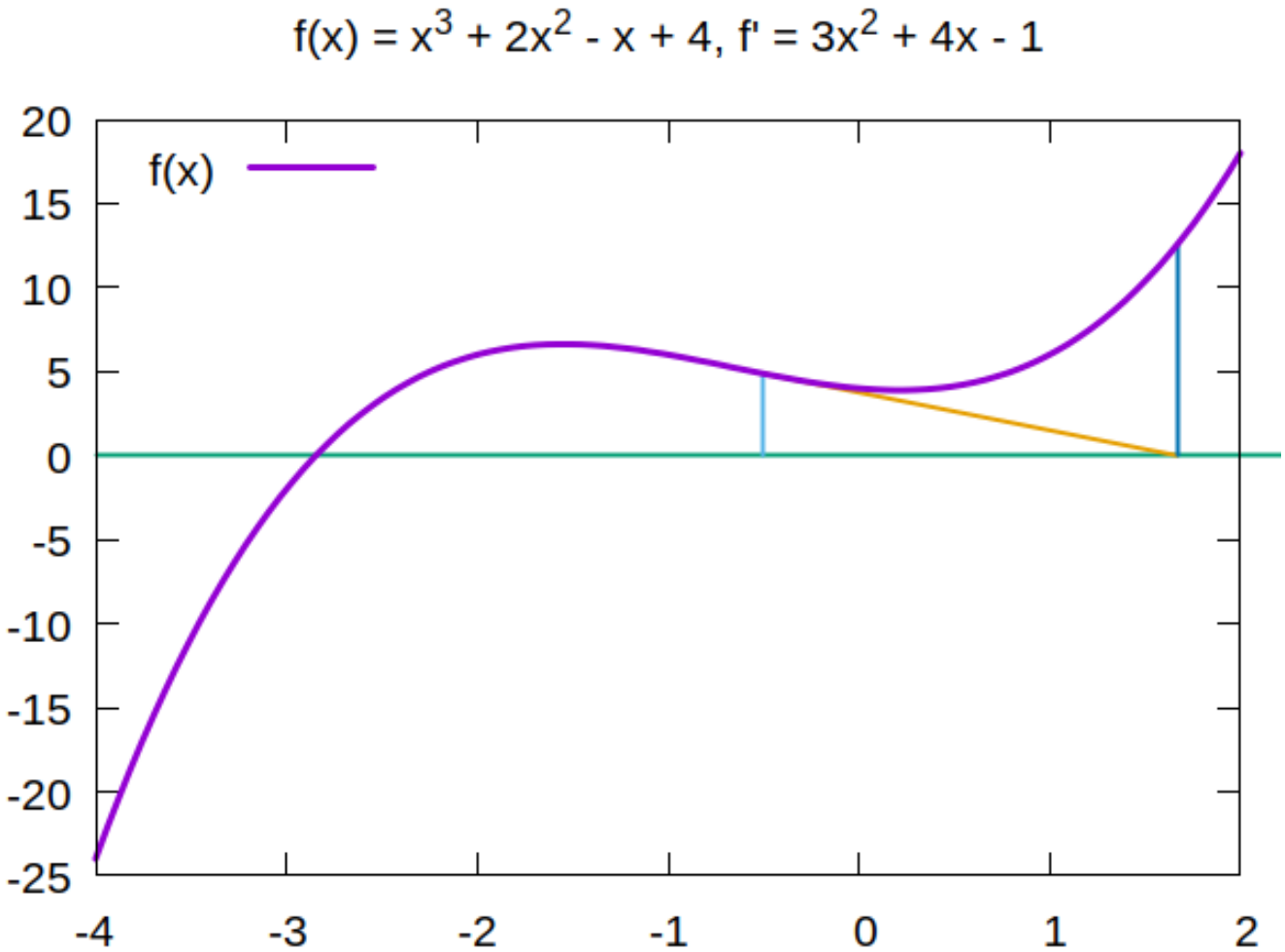
# Newton's method - a bad choice



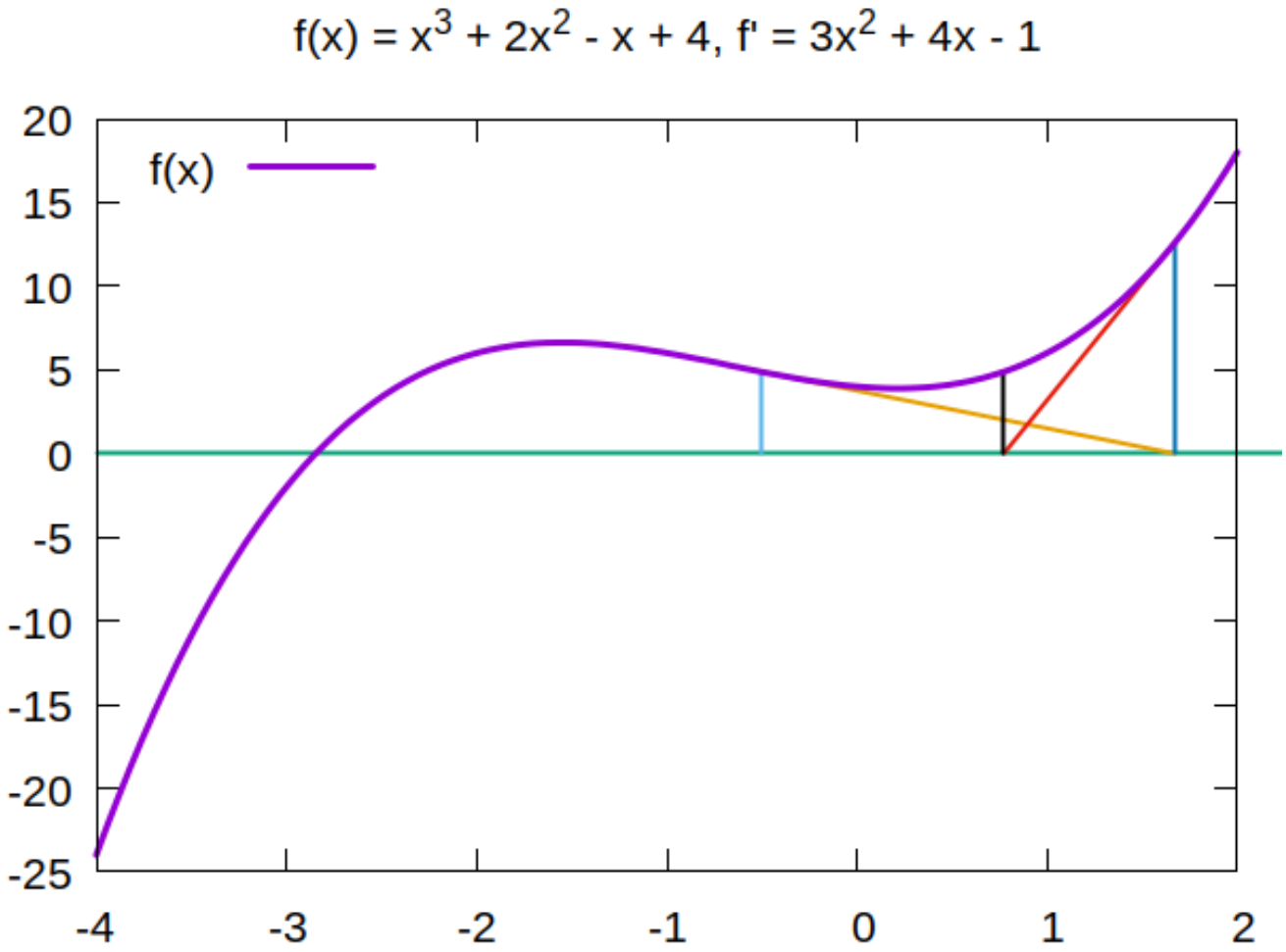
# Newton's method - a bad choice



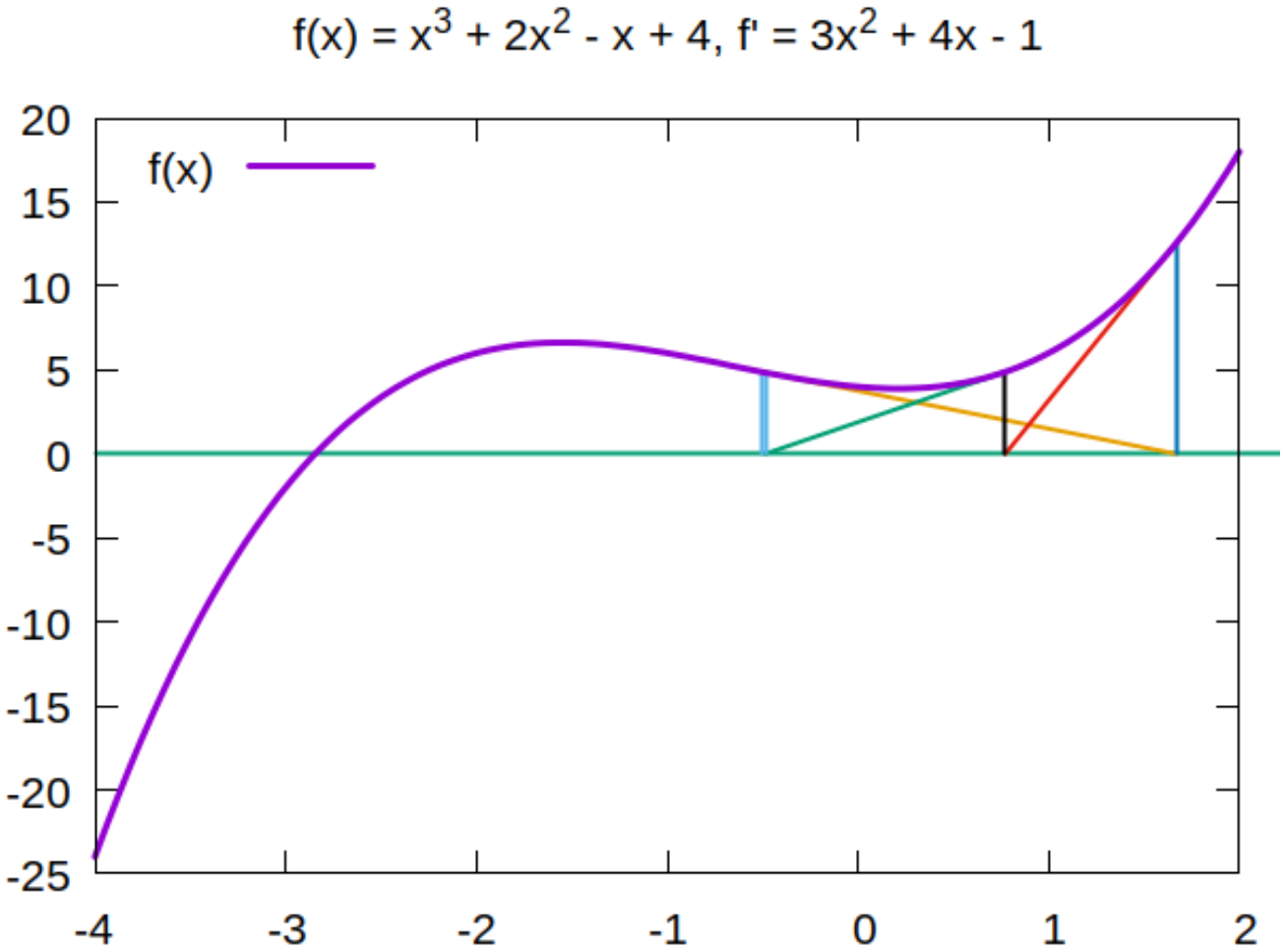
# Newton's method - a bad choice



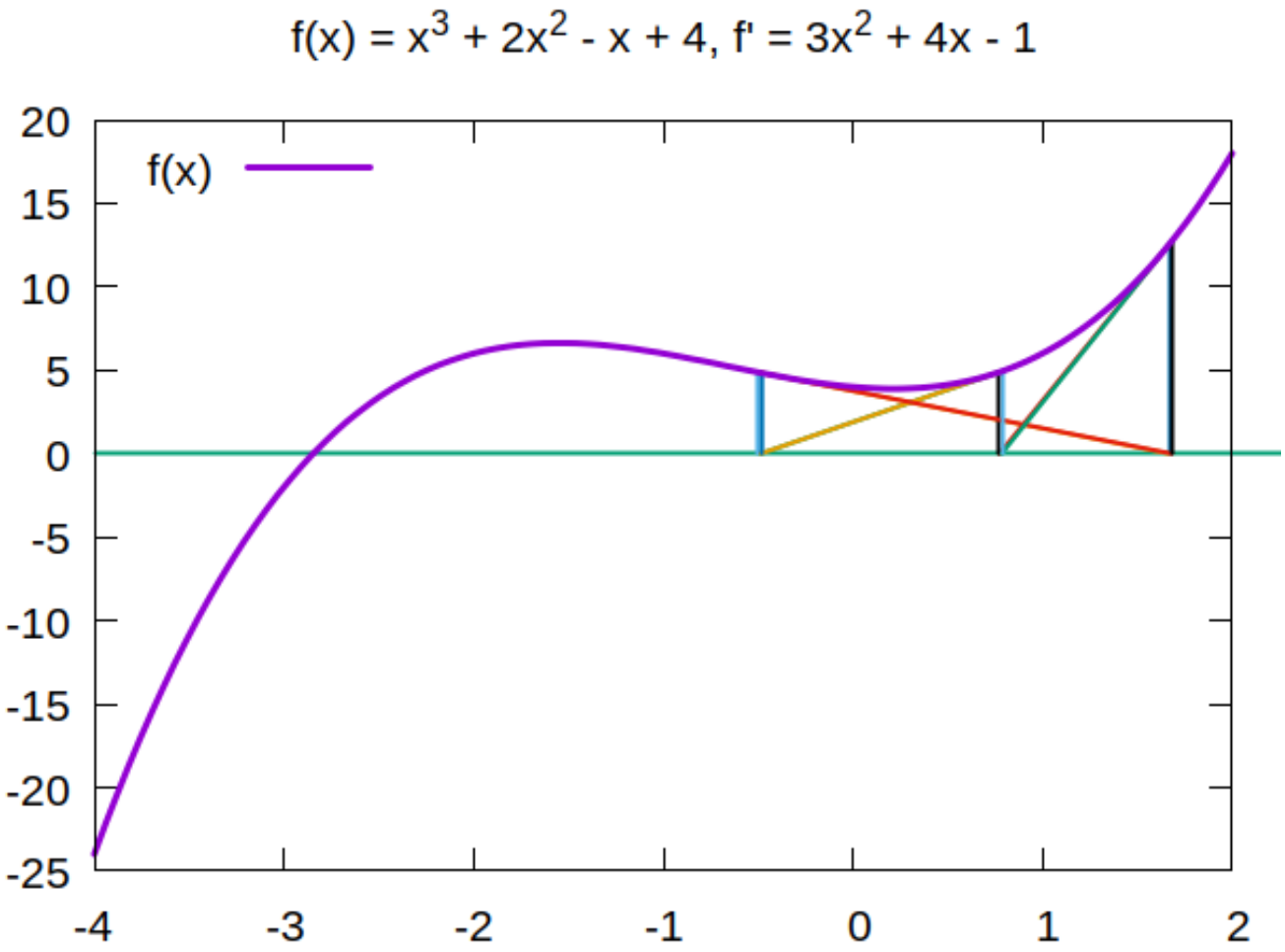
# Newton's method - a bad choice



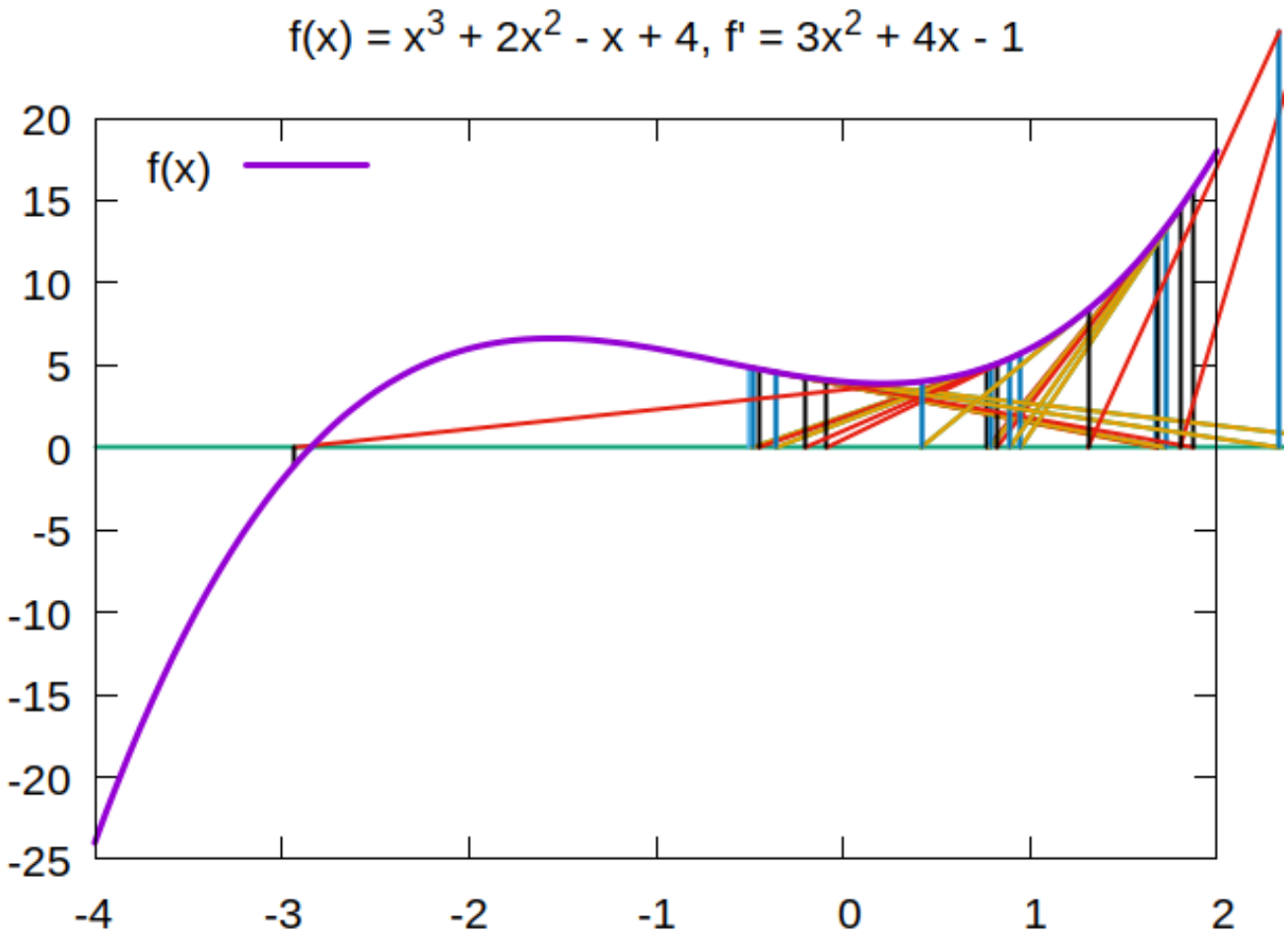
# Newton's method - a bad choice



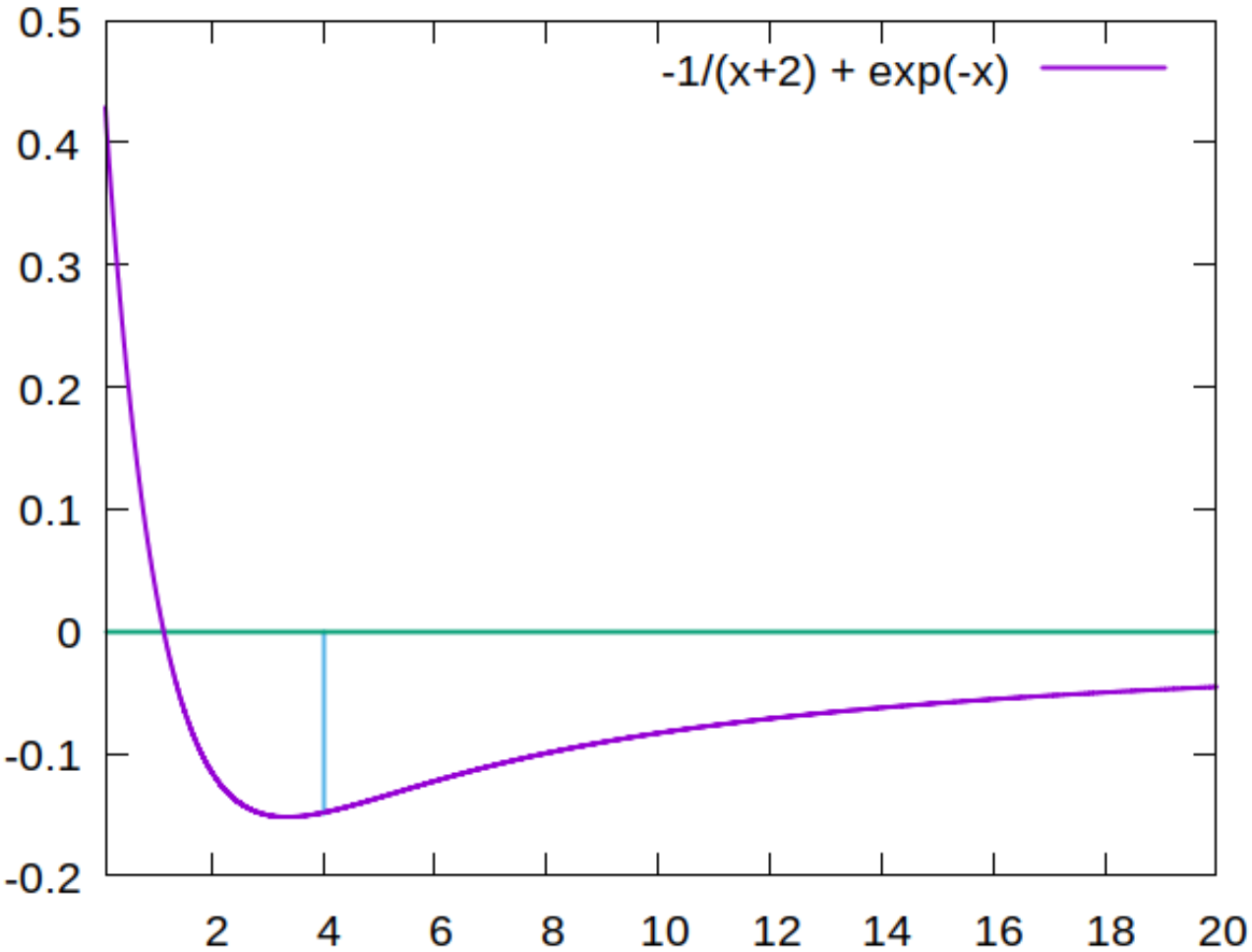
# Newton's method - a bad choice



# Newton's method - a bad choice



# Newton's method - a worse choice



# Outline


1. Discretisation and error
2. Fourier Series (briefly)
3. Interpolation
4. Root finding
5. Runge-Kutta methods

# Runge-Kutta methods

- Much of the scientific computing requires us to find solutions to partial differential equations, e.g.

$$\frac{\partial u_i}{\partial t} + \nabla_j f_{ij}\left(\mathbf{u}\right) = 0$$

- These techniques will often follow the **method of lines**, in which an approximation to the space derivative is made first, and this is used to solve the time derivative
- A lot of effort focuses on the space derivative, and in doing so, deals with the really complex aspects of the system
- There are then many fewer methods to required to deal with the time derivative, though they do tend to get glossed over
- Runge-Kutta methods are perhaps the most useful

# Runge-Kutta methods

$$\frac{\partial u_i}{\partial t} + \nabla_j f_{ij}\left(\mathbf{u}\right) = 0$$

- We shall assume that we know $F_i(\mathbf{u}) = -\nabla_j f_{ij}\left(\mathbf{u}\right)$
- At which point, each entry in $\mathbf{u}$ is now independent; we shall drop the index
- Runge-Kutta methods are designed to solve

$$\frac{\mathrm{d}u}{\mathrm{d}t} = F(u,t), \quad u(0) = u_0$$

- They are an iterative method,

$$u_{n+1} = u_n + h\sum_{i=1}^{s} b_i k_i$$

# Runge-Kutta methods

$$u_{n+1} = u_n + h \sum_{i=1}^{s} b_i k_i$$

- We assume that at a give time, $t_n$, we know the solution, $u_n = u(t_n)$
- We want to know the solution at a future time, $u_{n+1}$ which is an estimate of $u(t_{n+1}) = u(t_n + h)$
- $b_i$ are a series of constant coefficients
- $k_i$ are a series of successive evaluations of $F(u, t)$, with additional constants $c_i$ and $a_{ij}$

$$k_1 = F(u_n, t_n)$$
$$k_2 = F(u_n + h(a_{21}k_1), t_n + hc_2)$$
$$k_3 = F(u_n + h(a_{31}k_1 + a_{32}k_2), t_n + hc_3)$$
$$\vdots$$
$$k_s = F(u_n + h(a_{s1}k_1 + a_{s2}k_2 + \ldots + a_{s,s-1}k_{s-1}), t_n + hc_s)$$

# Euler method

- Identifying constants for a Runge-Kutta method is not always straightforward - there may be accuracy or efficiency issues
- However, some of the simpler methods can be derived fairly straightforwardly
- Recall our first-order derivative approximation (with $h = \Delta x$)

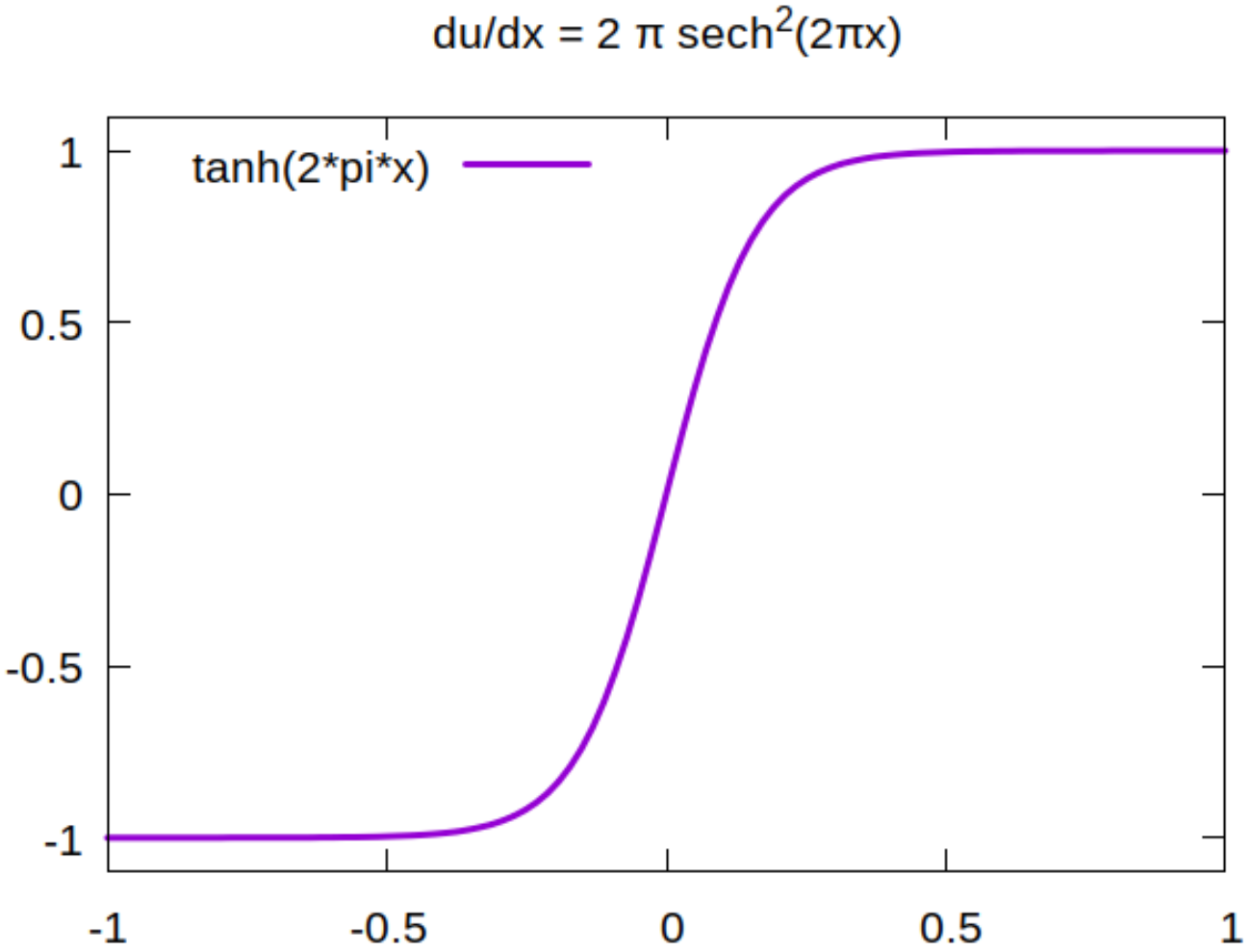
$$\frac{\partial u}{\partial t} \approx \frac{u(t+h) - u(t)}{h}$$

- Simple substitution gives

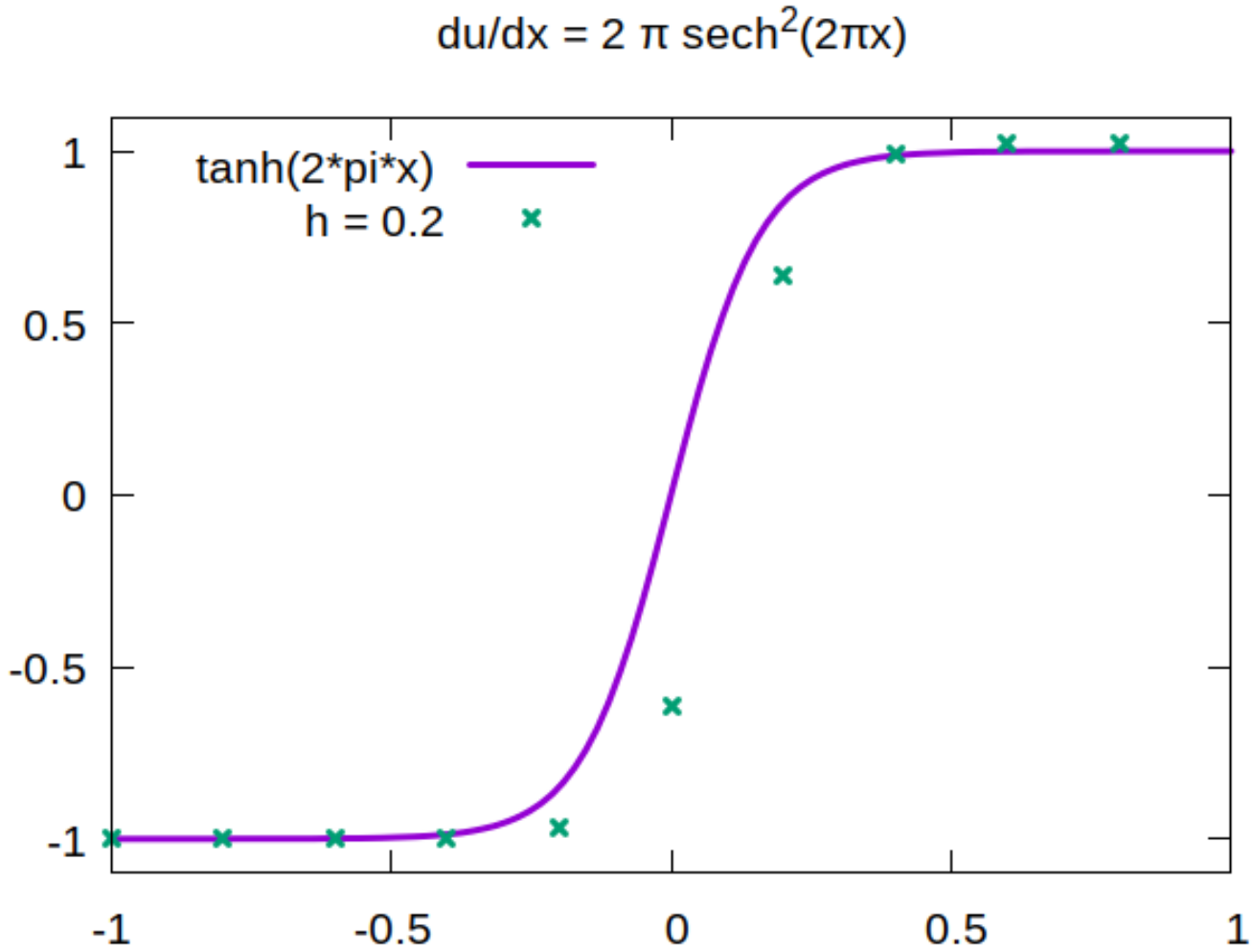
$$u_{n+1} = u_n + hF(u_n, t_n)$$

- Effectively, we assume the solution at $t_{n+1}$ is a straight line based on the derivative, $F(u, t)$

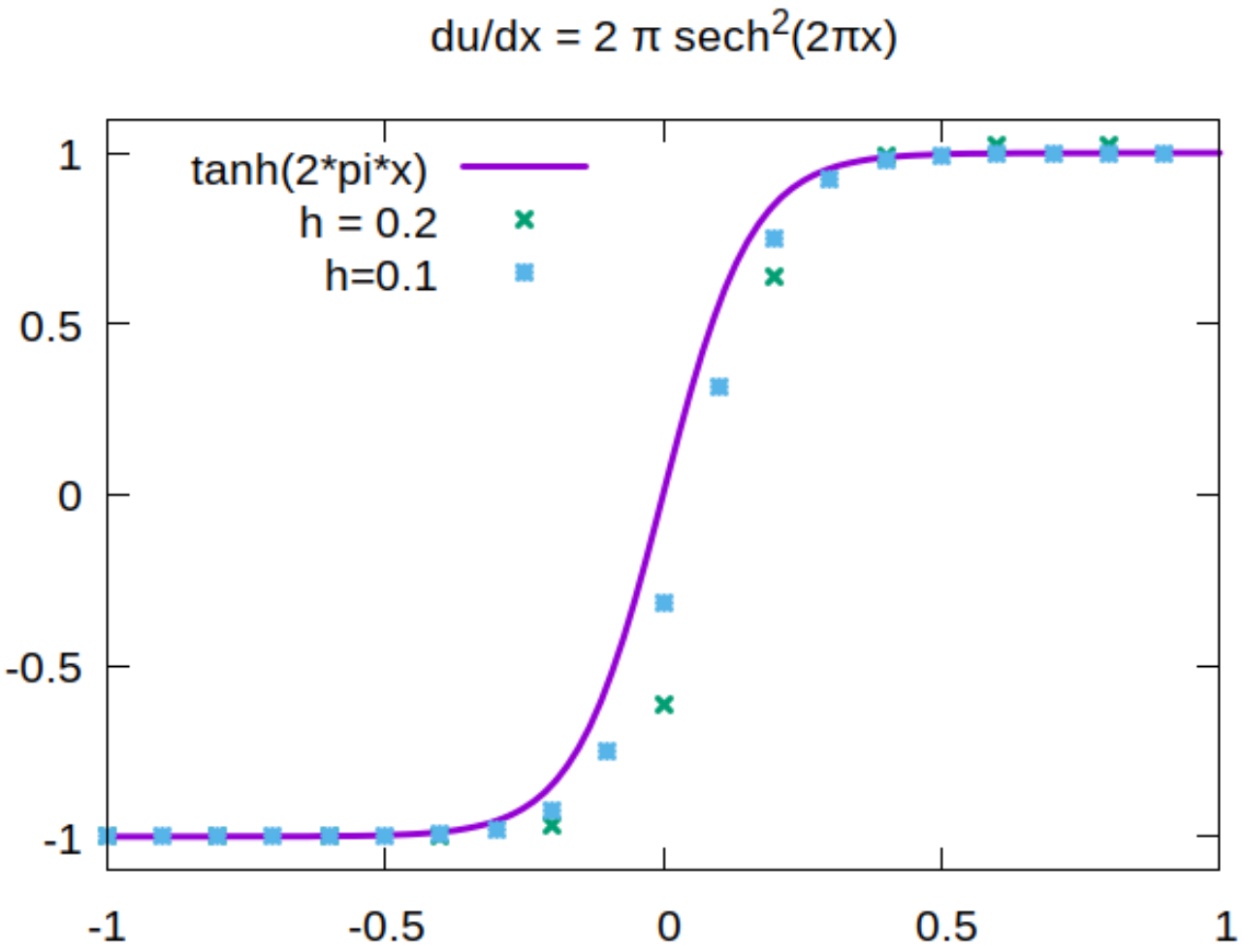
# Euler method



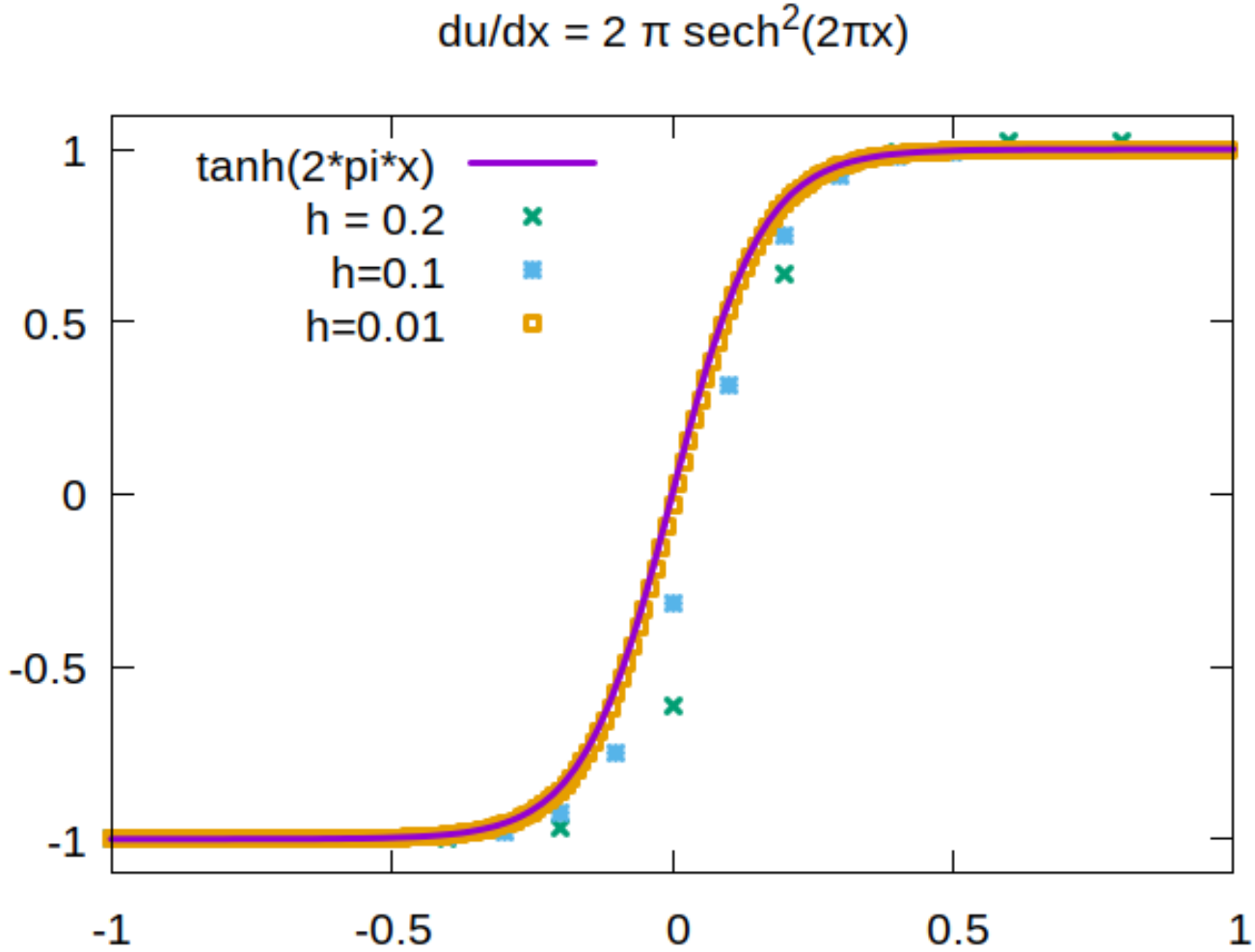
# Euler method



# Euler method



# Euler method



# Convergence of the Euler method

- Clearly, for this example, the Euler method worked. But it is more useful to show that this is general behaviour - that the method **converges**
- We know that technically, we have

$$u_{n+1} = u_n + hF(u_n, t_n) + \mathcal{O}(\Delta h)$$

- Convergence shows that these additional terms don't build up until; i.e. error doesn't grow unboundedly
- We can show that, for suitable functions, $F(u, t)$, the **total error** has a bound that is proportional to $h$,

$$|E| = \sum_{i=0}^{N} |E_i| \leq bh + \mathcal{O}(h^2)$$

# Lipschitz continuity

- When we say that $F(u,t)$ must be a suitable function, we mean that it is **Lipschitz continuous**
- For a Lipschitz continuous function, $f(x)$, a constant, $K$, exists such that

$$|f(x_2) - f(x_1)| \leq K\,|x_2 - x_1|$$

- Roughly speaking, the derivative of $f$ must be bounded - this is quite a reasonable expectation in physical systems
- If this is the case, we can show that

$$|E|_{n+1} = u_{n+1} - u(t_{n+1}) \leq (1 + hK)\,|E|_n + ah^2$$

- And from this definition, we can show

$$|E| \leq bh + \mathcal{O}(h^2)$$

# Midpoint method

- By starting with a second-order accurate derivative approximation, the **midpoint method** is reached

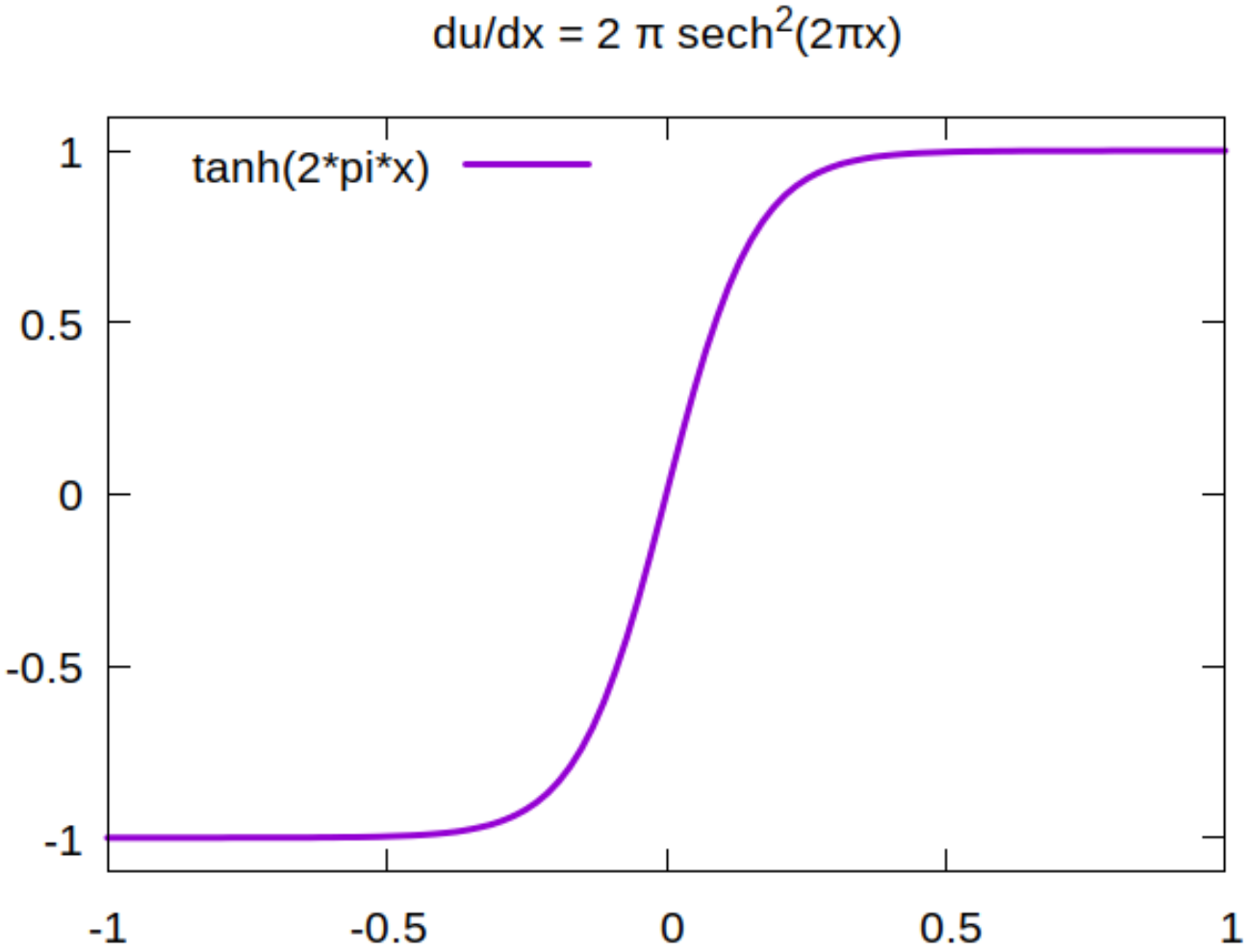
$$u_{n+1} = u_n + hF\left(u_n + \frac{h}{2}F(u_n, t_n), t_n + \frac{h}{2}\right)$$

- This can be thought of as a two-step process
- First, we approximate the derivative at $t + h/2$,

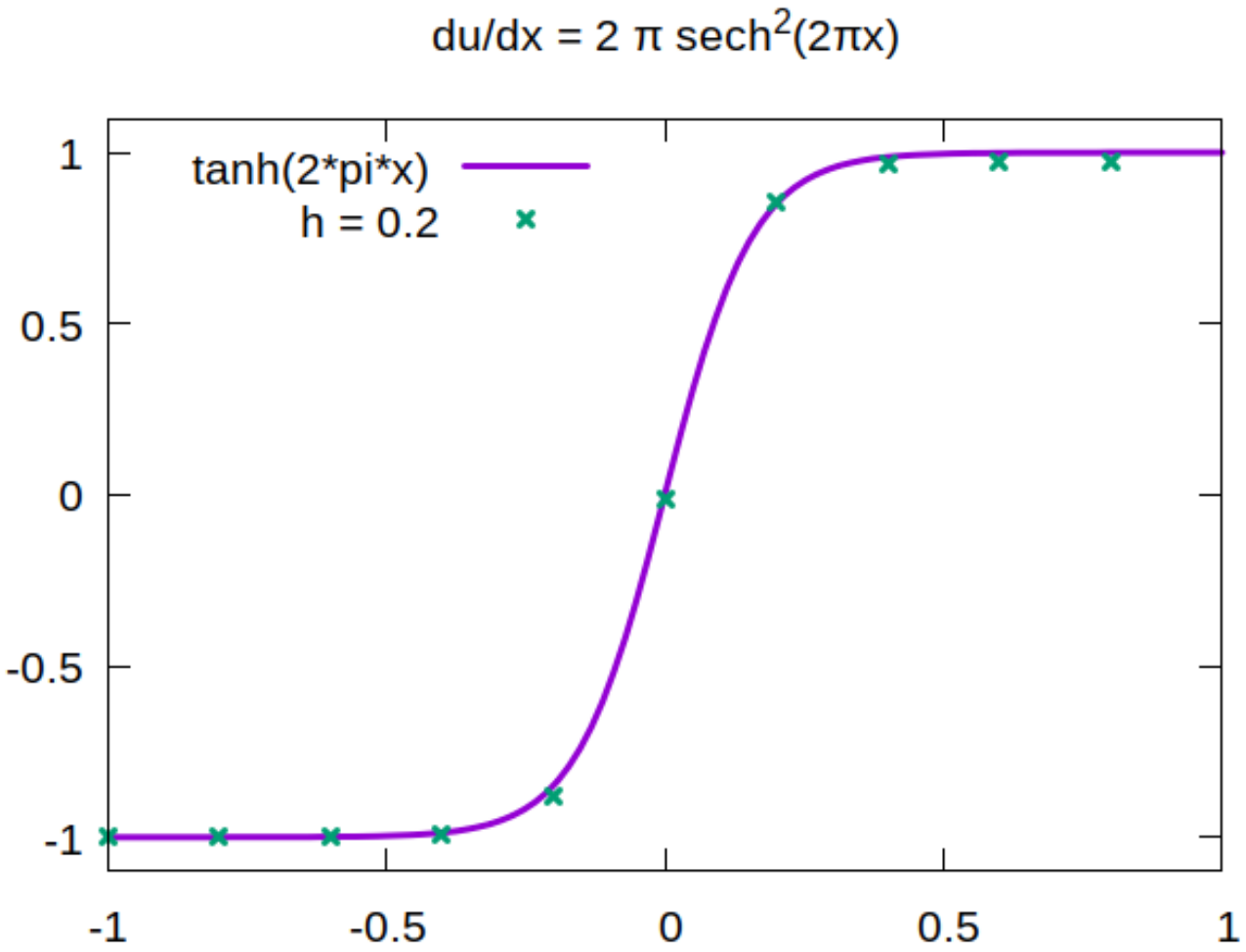
$$F\left(u_n + \frac{h}{2}F(u_n, t_n), t_n + \frac{h}{2}\right)$$

- Then we use this approximation to advance to $t_{n+1}$ in the same manner as we did for the Euler method

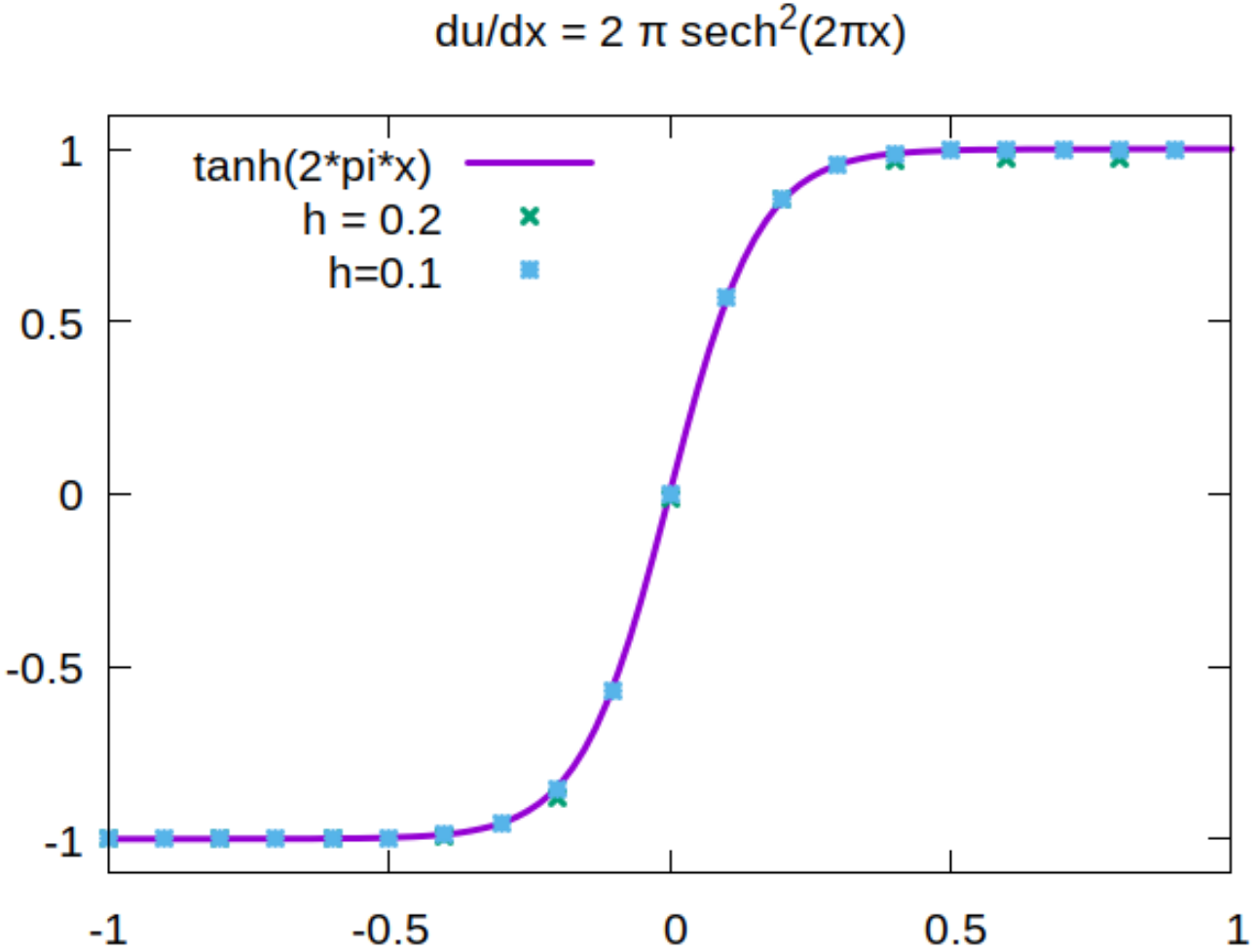
# Midpoint method



# Midpoint method



# Midpoint method



# Butcher tableaux

$$u_{n+1} = u_n + h \sum_{i=1}^{s} b_i k_i$$

$$k_1 = F(u_n, t_n)$$

$$k_2 = F(u_n + h(a_{21} k_1), t_n + h c_2)$$

$$\vdots$$

$$k_s = F(u_n + h(a_{s1} k_1 + a_{s2} k_2 + \ldots + a_{s,s-1} k_{s-1}), t_n + h c_s)$$

- The methods considered so far fit into this form
- For the Euler method, we have $b_1 = 1$
- For the midpoint method, we have $b_1 = 0$, $b_2 = 1$, $a_{21} = \frac{1}{2}$ and $c_2 = \frac{1}{2}$
- There are many other Runge-Kutta methods, in order to make things slightly easier to read, these constants can be written in a **Butcher tableau**

# Butcher tableaux

$$u_{n+1} = u_n + h \sum_{i=1}^{s} b_i k_i$$

$$k_1 = F(u_n, t_n)$$

$$k_2 = F(u_n + h(a_{21} k_1), t_n + h c_2)$$

$$\vdots$$

$$k_s = F(u_n + h(a_{s1} k_1 + a_{s2} k_2 + \ldots + a_{s,s-1} k_{s-1}), t_n + h c_s)$$

$$\begin{array}{c|ccccc} 0 & & & & & \\ c_2 & a_{21} & & & & \\ c_3 & a_{31} & a_{32} & & & \\ \vdots & \vdots & & \ddots & & \\ c_s & a_{s1} & a_{s2} & \ldots & a_{s,s-1} & \\ \hline & b_1 & b_2 & \ldots & b_{s-1} & b_s \end{array}$$

# Some Runge-Kutta examples

- Euler method (first-order accurate):

$$\begin{array}{c|c} 0 & \\ \hline & 1 \end{array}$$

- Midpoint method (second-order accurate):

$$\begin{array}{c|cc} 0 & & \\ \frac{1}{2} & \frac{1}{2} & \\ \hline & 0 & 1 \end{array}$$

- Heun's method (second-order accurate)

$$\begin{array}{c|cc} 0 & & \\ 1 & 1 & \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array}$$

- **The** (original) Runge-Kutta method (fourth-order accurate):

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}$$

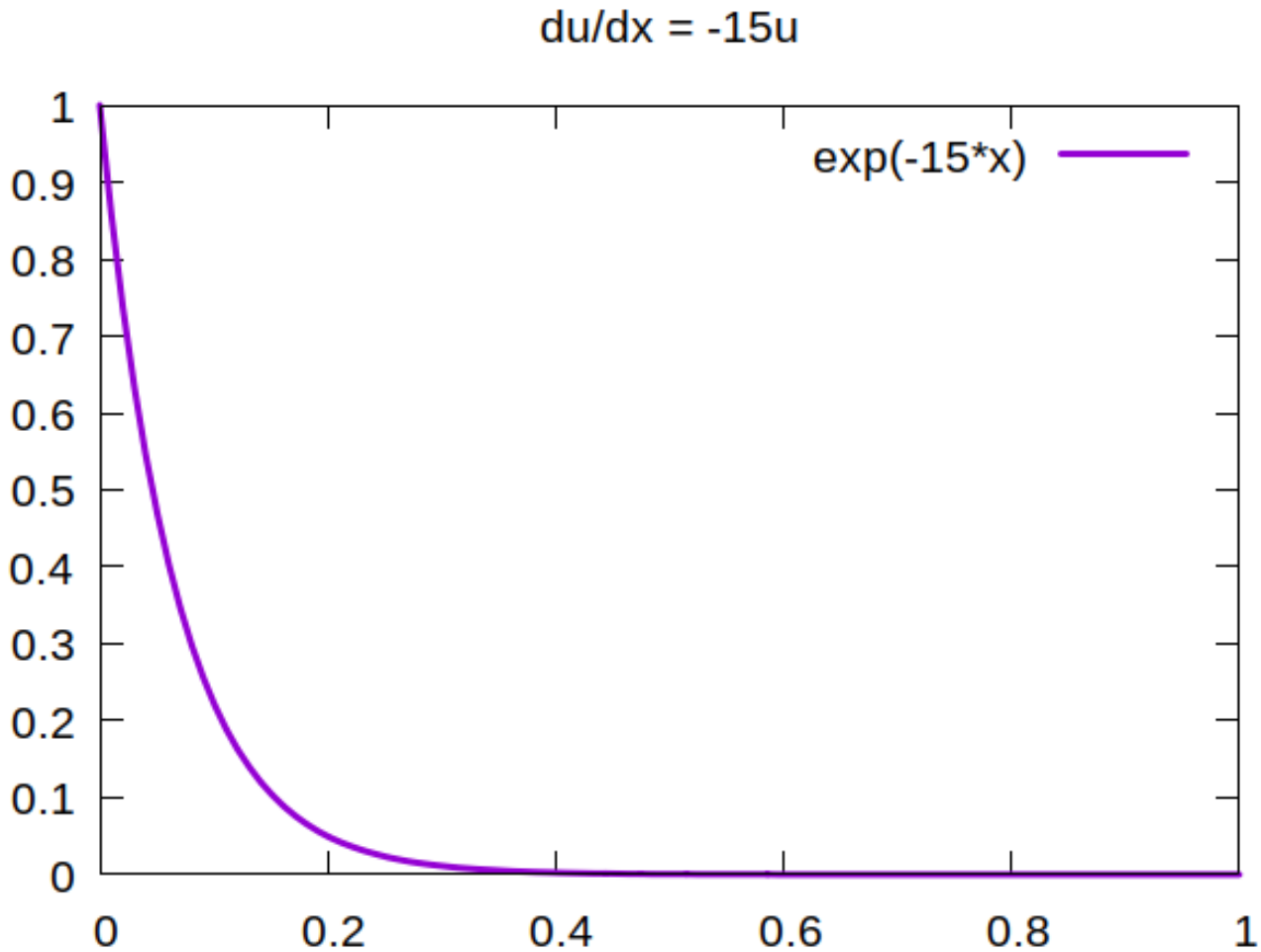
- 3/8-rule (fourth-order accurate):

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{3} & \frac{1}{3} & & & \\ \frac{2}{3} & -\frac{1}{3} & 1 & & \\ 1 & 1 & -1 & 1 & \\ \hline & \frac{1}{8} & \frac{3}{8} & \frac{3}{8} & \frac{1}{8} \end{array}$$

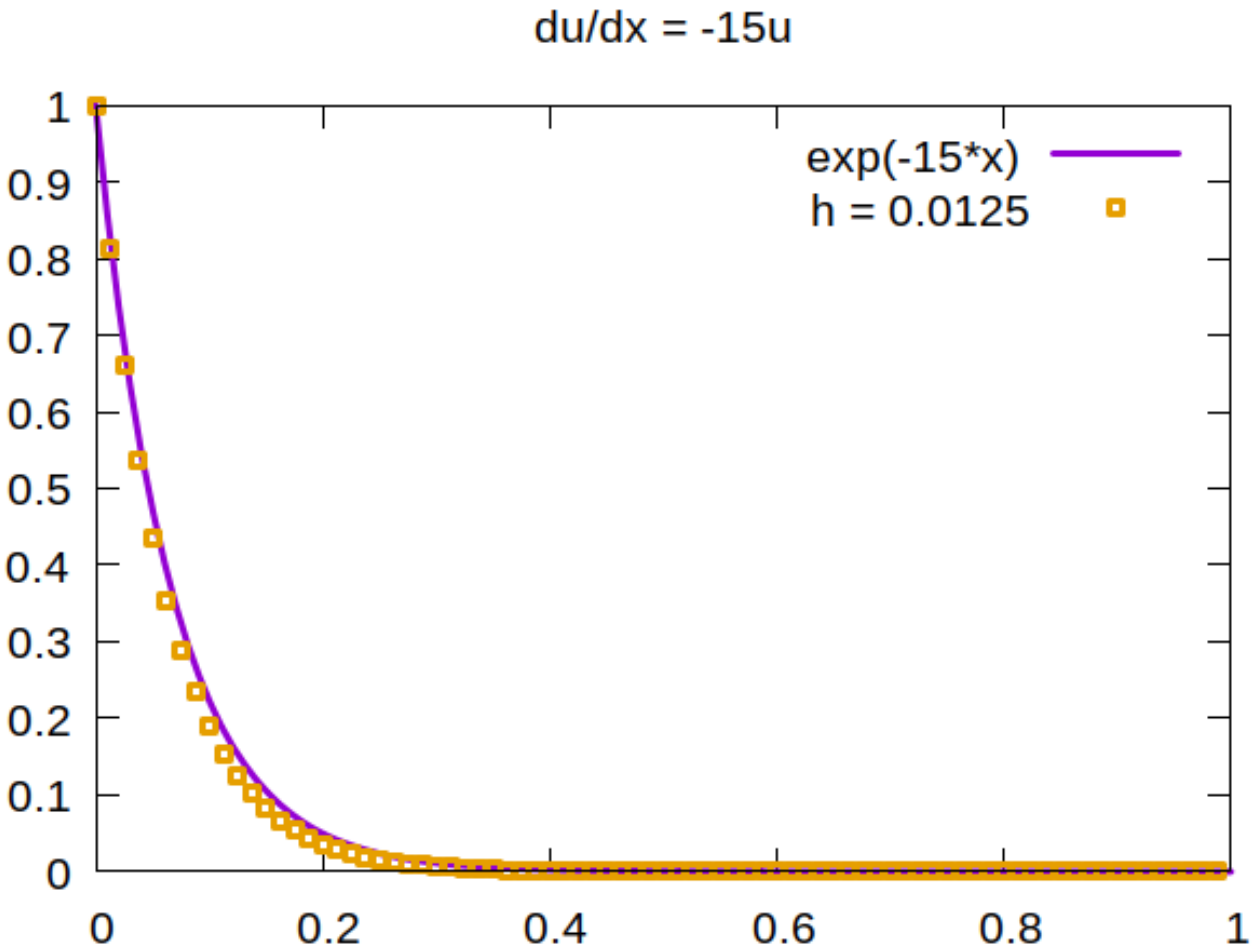
# Why are there so many?

- As with any numerical method, the choice of Runge-Kutta method used is a balance between accuracy, efficiency and stability
- Runge-Kutta methods are often applied as part of a numerical method for PDE's, ideally the accuracy of the numerical method and the RK method should be the same
- The 3/8 rule has lower **absolute error** than the original Runge-Kutta method
- However, the original method can be written in such a way that $k_1$, $k_2$ etc. do not need to be stored
- This can represent a large saving in computational memory (if an array of $k$ values is needed)
- If a Runge-Kutta method does not produce a good solution (for example, it oscillates wildly), this could be because it is **unstable**
- We shall consider stability later, but for these methods, this stability can be dependent on the form of $F(u, t)$

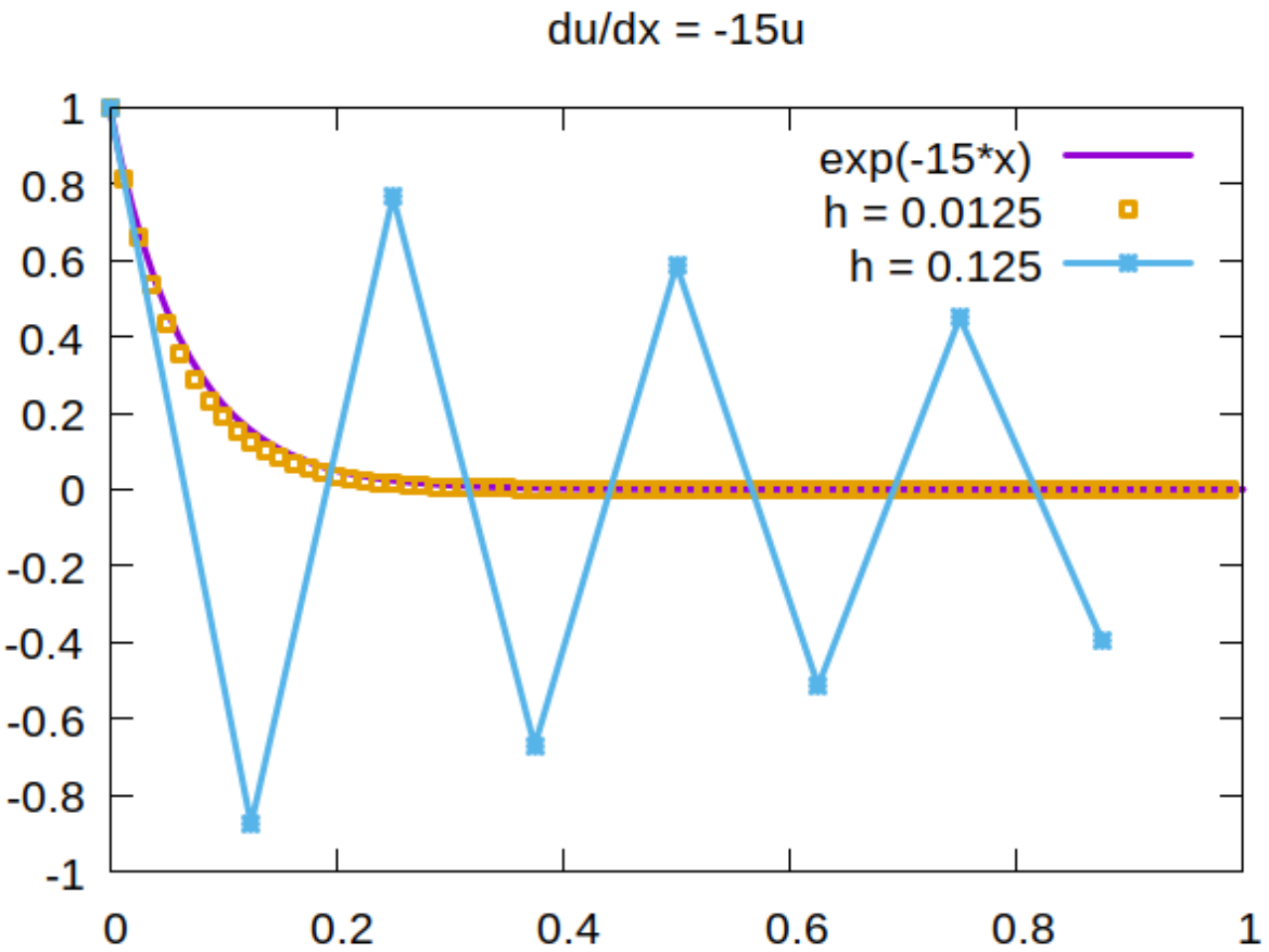
# Unstable behaviour



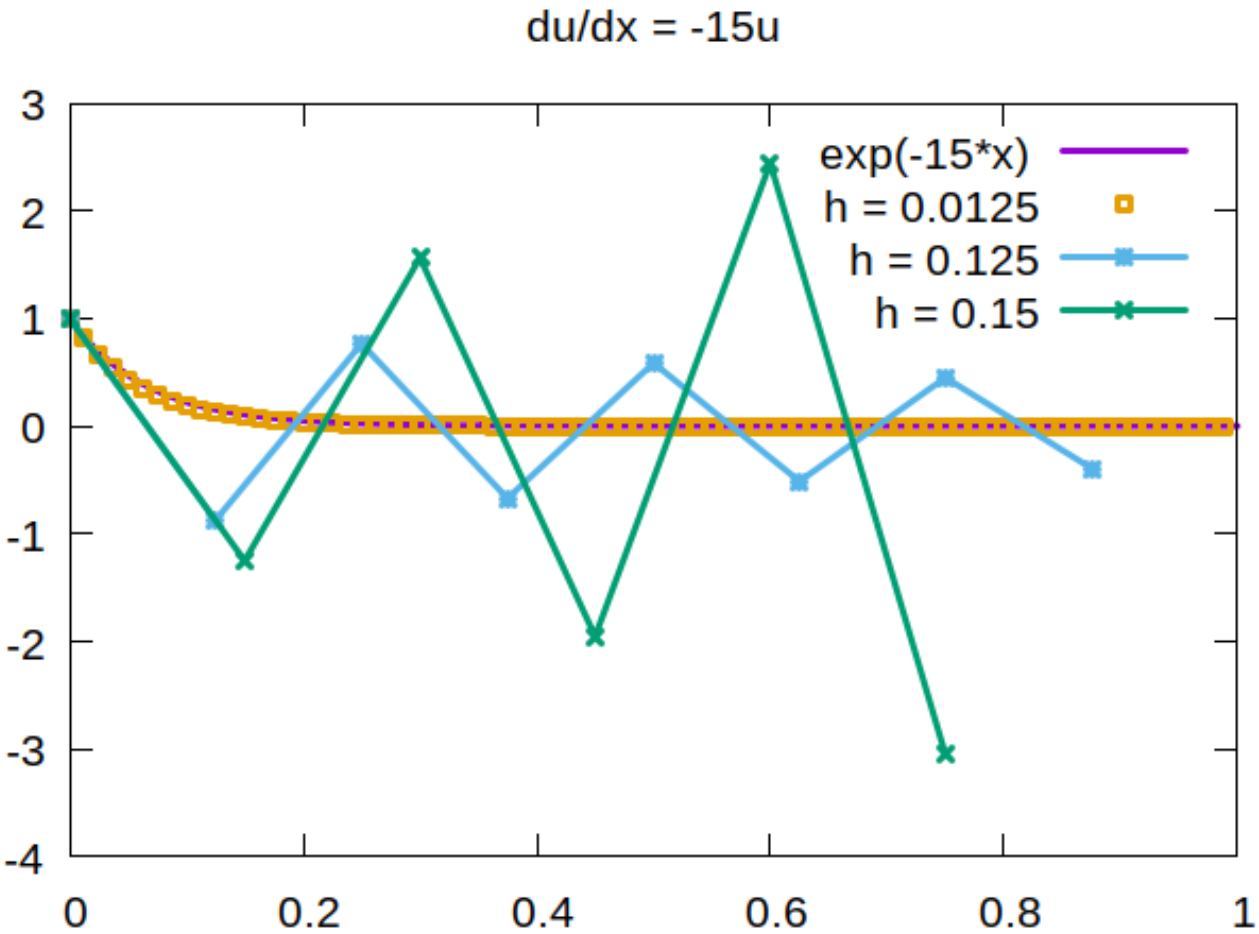
# Unstable behaviour



# Unstable behaviour



# Unstable behaviour



# Summary

- We have covered some of the underlying mathematics which will appear repeatedly throughout the continuum courses
- Some of these will be built upon in greater detail, e.g. thermodynamics for equations of state, and discretisation techniques
- Anything in this course should be considered examinable - these methods could be required in any of the continuum exams
- However, this will always be in context of the course itself, it will not come as a surprise question based only on these lectures